Unit 3: Two-Dimensional Geometric Transformations (5 Hrs.)

-By P.B.Jha

3.1 Two-Dimensional translation, Rotation, Scaling, Reflection and Shearing

3.2 Homogeneous Coordinate and 2D Composite Transformations. Transformation between Co-ordinate Systems.

3.3 Two Dimensional Viewing: Viewing pipeline, Window to viewport coordinate transformation

3.4 Clipping: Point, Lines(Cohen Sutherland line clipping, Liang-Barsky Line Clipping) ,Polygon Clipping(Sutherland Hodgeman polygon clipping)

- transformations help change the object's position, size, orientation, shape, etc.;
- there are three basic rigid transformations: reflections, rotations, and translations.
- 2D Transformations in computer graphics take place in a two-dimensional plane.
- Transformations play an important role in computer graphics to reposition the graphics on the screen and change their size or orientation.

What is 2D Transformation in Computer Graphics?

- Transformation means a change in the object's orientation, size, and shape.
- They position the object, change its shape, and even change how something is viewed.
- Transformation plays a major role in computer graphics, repositioning the graphics on the screen and changing their size or orientation.
- The basic geometrical 2D Transformation in Computer Graphics are:

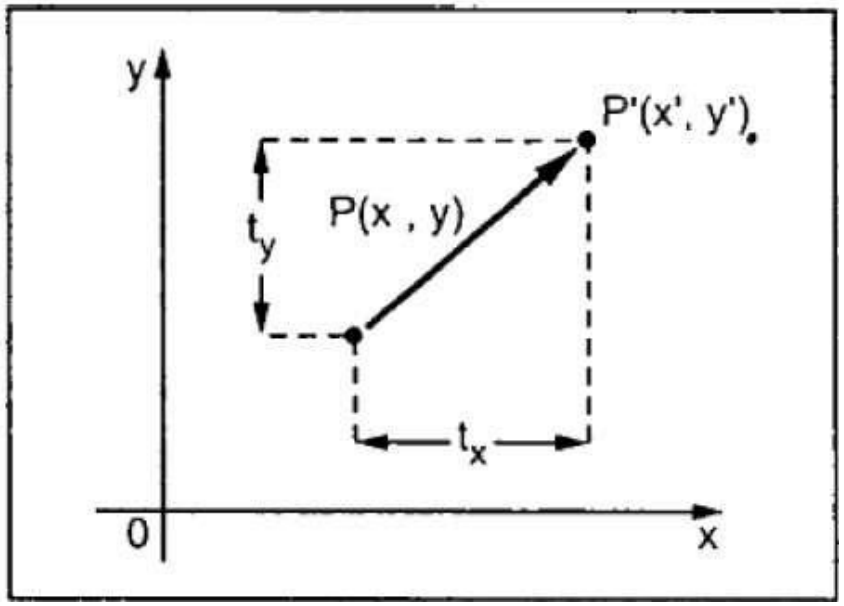
- Translation
- Rotation
- Scaling

The derived geometrical transformation is:

- Reflection
- Shearing

❑ Translation

- A translation moves an object to a different position on the screen.
- You can translate a point in 2D by adding translation coordinate (tx, ty) to the original coordinate X,Y
- to get the new coordinate X',Y'
- .



From the above figure, you can write that –

$X' = X + t_x$

$Y' = Y + t_y$

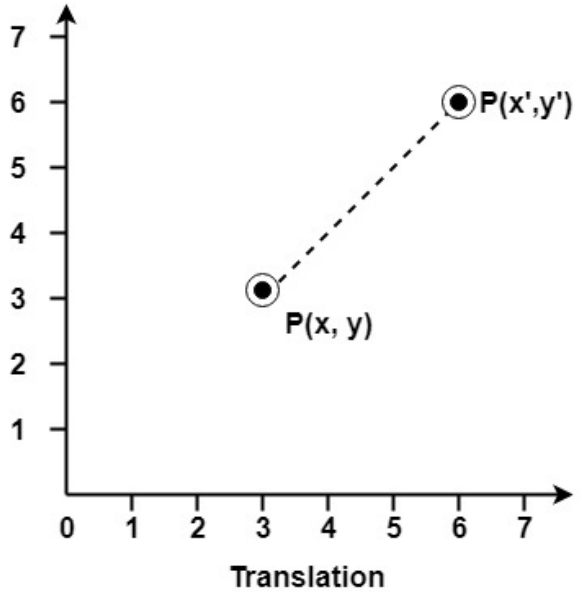
The pair (tx, ty) is called the translation vector or shift vector. The above equations can also be represented using the column vectors.

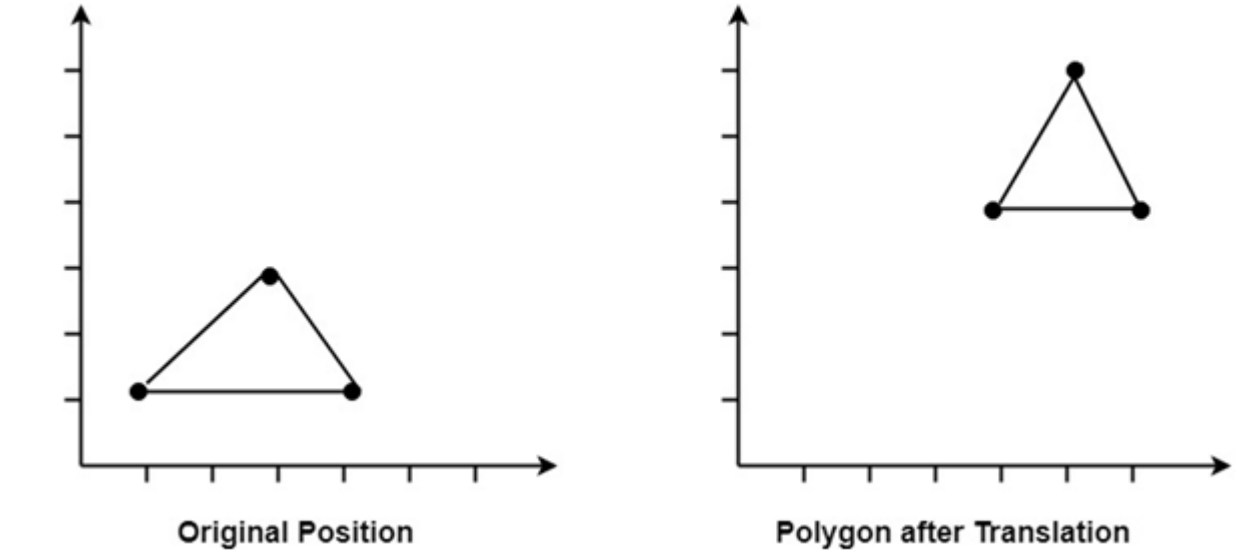
$$P = \frac{[X]}{[Y]} \qquad p' = \frac{[X']}{[Y']} \qquad T = \frac{[t_x]}{[t_y]}$$

We can write it as –

$$P' = P + T$$

Let P is a point with coordinates (x, y). It will be translated as (x^1 y^1).

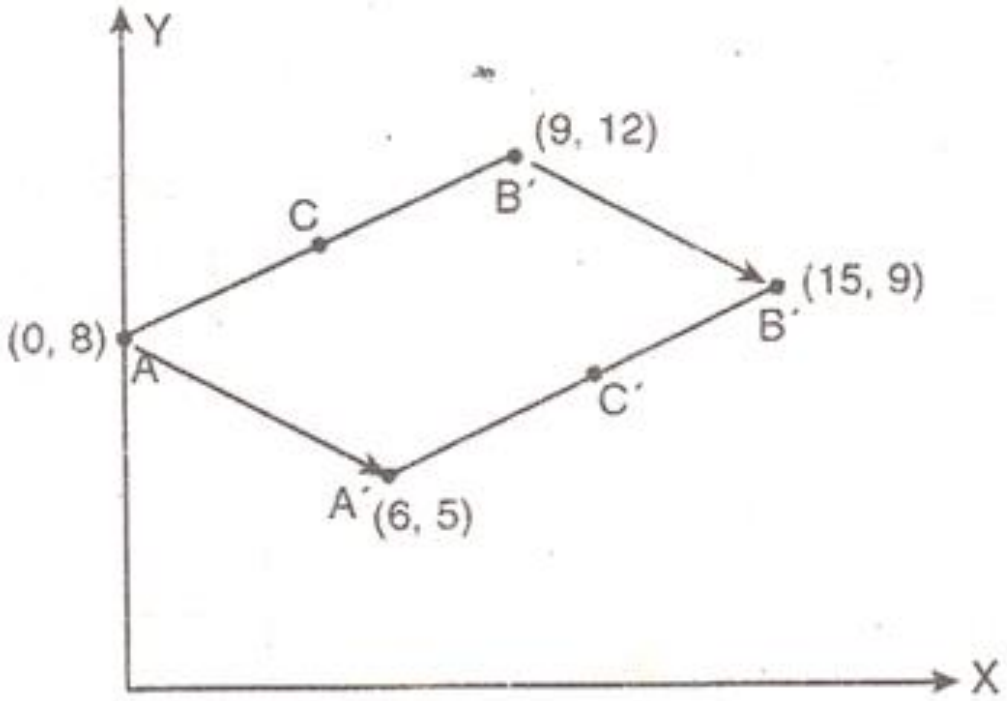




Matrix for Translation:

$$\begin{vmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ t_x & t_y & 1 \end{vmatrix} \quad \text{Or} \quad \begin{vmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{vmatrix}$$

K=5



❑Rotation:

- It is a process of changing the angle of the object.
- Rotation can be clockwise or anticlockwise. For rotation, we have to specify the angle of rotation and rotation point. Rotation point is also called a pivot point.
- It is print about which object is rotated.

❑ Types of Rotation:

a. Clockwise [-ve]
b. Anti clockwise direction [+ve]

- The positive value of the pivot point (rotation angle) rotates an object in a counter-clockwise (anti-clockwise) direction.

- The negative value of the pivot point (rotation angle) rotates an object in a clockwise direction.

- When the object is rotated, then every point of the object is rotated by the same angle.

1. Straight Line: Straight Line is rotated by the endpoints with the same angle and redrawing the line between new endpoints.

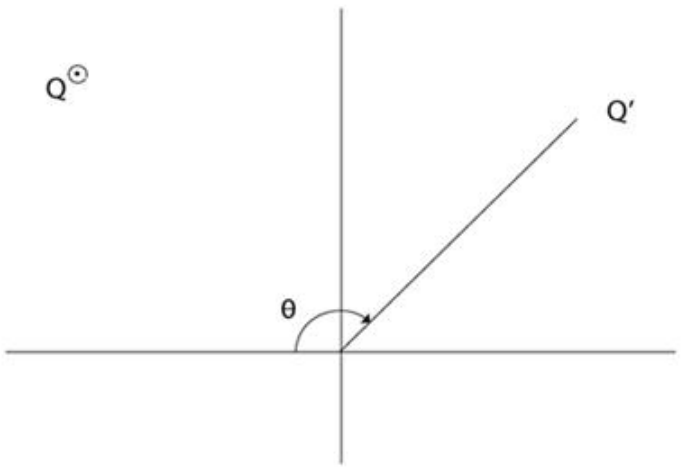
2. Polygon: Polygon is rotated by shifting every vertex using the same rotational angle.

3. Curved Lines: Curved Lines are rotated by repositioning of all points and drawing of the curve at new positions.

4. Circle: It can be obtained by center position by the specified angle.

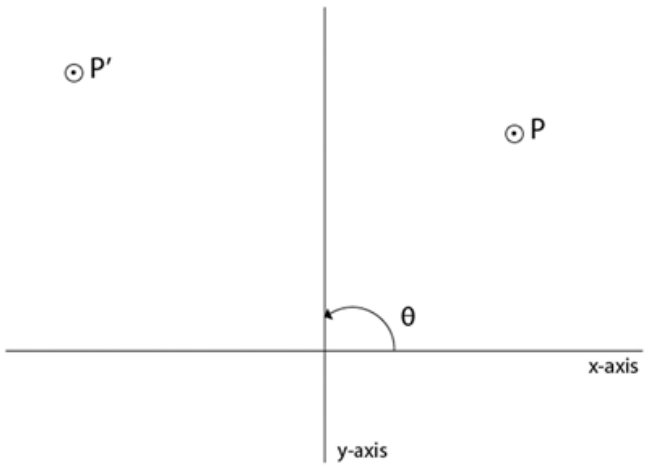
5. Ellipse: Its rotation can be obtained by rotating major and minor axis of an ellipse by the desired angle

Rotation in anticlockwise direction



Q is original position
Q ' is final rotated position

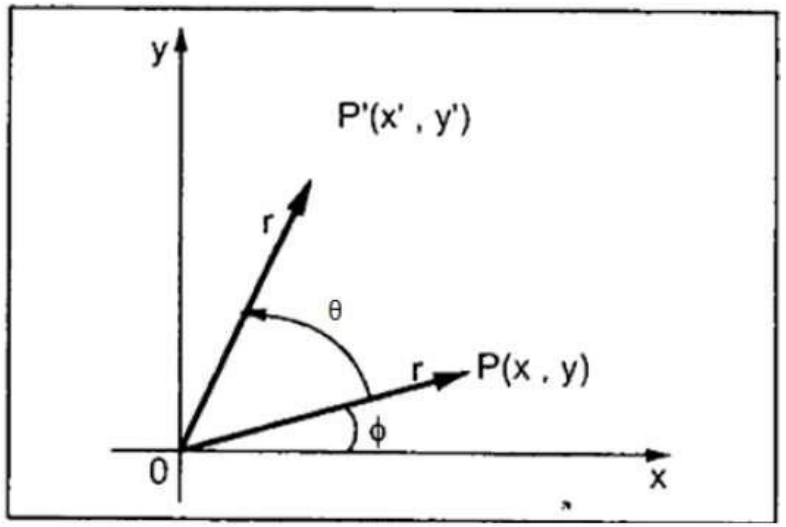
Rotation of P in clockwise direction

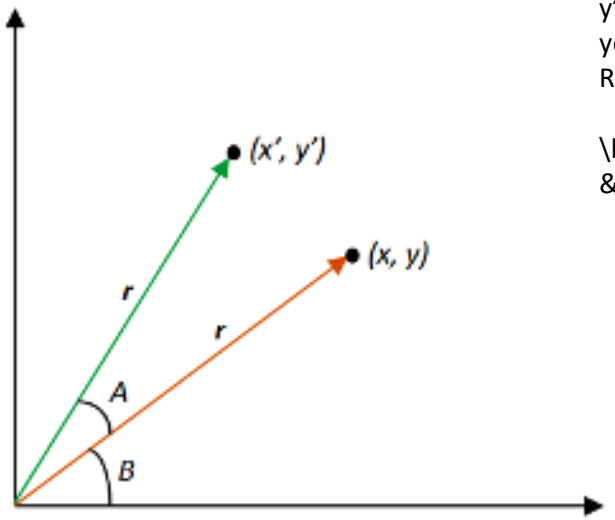


P is original Position
P' is final position or position after rotation
where θ is angle of rotation

❑ Rotation

- In rotation, we rotate the object at particular angle θ theta
- from its origin. From the following figure, we can see that the point P (X,Y) is located at angle φ from the horizontal X coordinate with distance r from the origin.
- Let us suppose you want to rotate it at the angle θ. After rotating it to a new location, you will get a new point P' X′,Y′





We know that,
x = rcosB, y = rsinB
x' = rcos(A+B) = r(cosAcosB – sinAsinB) = rcosBcosA – rsinBsinA = xcosA – ysinA
y' = rsin(A+B) = r(sinAcosB + cosAsinB) = rcosBsinA + rsinBcosA = xsinA + ycosA
Rotational Matrix Equation:-

\begin{bmatrix} x' \\ y' \end{bmatrix} =\begin{bmatrix} cosA & -sinA\\ sinA & cosA \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} cosA & -sinA \\ sinA & cosA \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

Using standard trigonometric the original coordinate of point PX,Y
can be represented as −

$X = r\cos\phi$.....(1)

$Y = r\sin\phi$.....(2)

Same way we can represent the point P'(X′,Y′)
as −

$x' = r\cos(\phi+\theta) = r\cos\phi\cos\theta - r\sin\phi\sin\theta$.....(3)

$y' = r\sin(\phi+\theta) = r\cos\phi\sin\theta + r\sin\phi\cos\theta$.....(4)

Substituting equation 1& 2 in 3& 4
respectively, we will get

$x' = x\cos\theta - y\sin\theta$
$y' = x\sin\theta + y\cos\theta$

Matrix Representation

$$P' = R \cdot P$$

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix}$$

ROTATION MATRX

The rotation angle can be positive and negative.

For positive rotation angle, we can use the above rotation matrix. However, for negative angle rotation, the matrix will change as shown below −

$$R = \begin{bmatrix} \cos\theta & \sin\theta \\ -\sin\theta & \cos\theta \end{bmatrix}$$

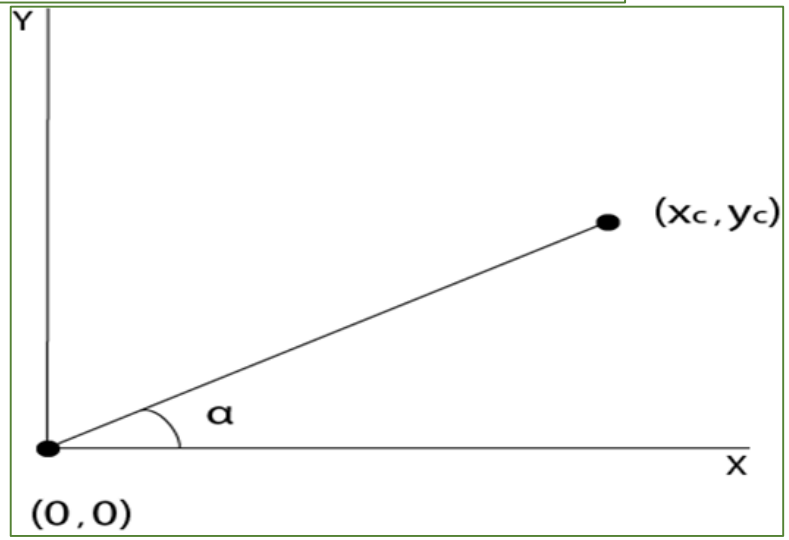
❑Rotation about an arbitrary point:

- If we want to rotate an object or point about an arbitrary point, first of all,
- we translate the point about which we want to rotate to the origin. Then rotate point or object about the origin, and at the end, we again translate it to the original place. We get rotation about an arbitrary point.

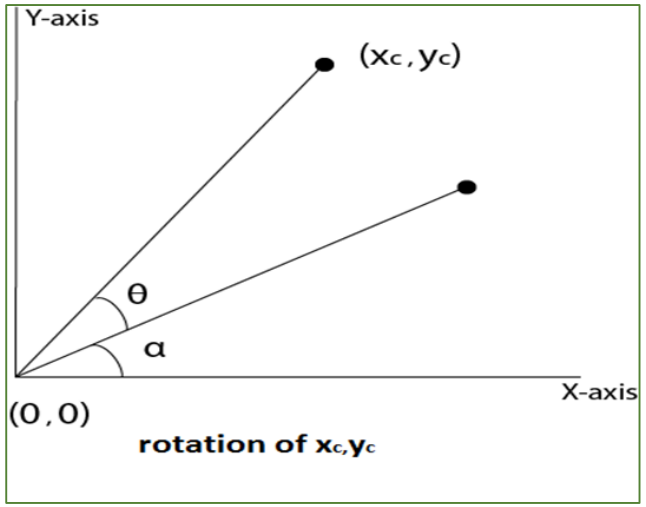
Example: The point (x, y) is to be rotated

The (xc yc) is a point about which anticlockwise rotation is done

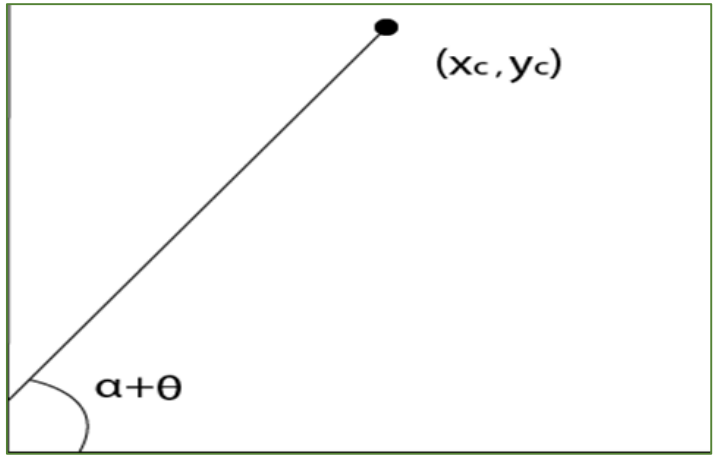
Step1: Translate point (xc yc) to origin

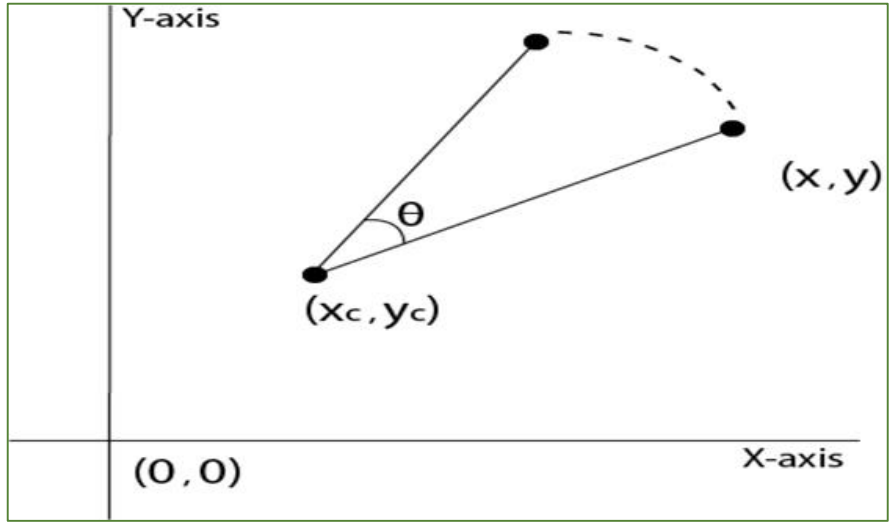


Step2: Rotation of (x, y) about the origin



Step3: Translation of center of rotation back to its original position





Rotation about arbitrary point:

Suppose the reference point of rotation is other than origin, then in that case we have to follow series of transformation.

Consider figure 17, assume that we have to rotate a point P1 with respect to (Xm, Ym) then we have to perform three steps.

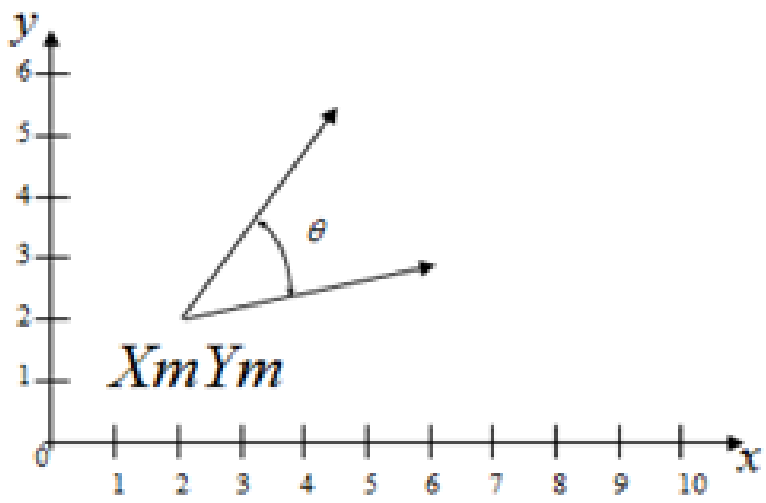


Figure 17

I) **Translation:** First we have to translate the (Xm, Ym) to origin as shown in figure 18. Translation matrix (T1) will become

$$T1 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ -Xm & -Ym & 1 \end{bmatrix}$$

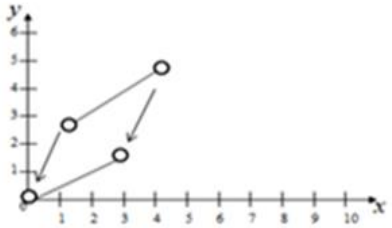


Figure 18

Here in figure 18, Tx = -Xm and Ty = -Ym.

II) **Rotation:** Rotate it in clockwise or anticlockwise direction. See figure 19. Let assume as anticlockwise rotation by angle θ. So our rotation matrix will be

$$R = \begin{bmatrix} Cos\theta & Sin\theta & 0 \\ -Sin\theta & Cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

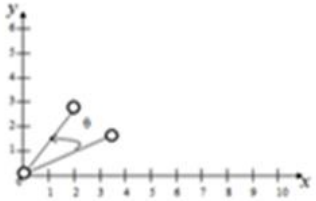


Figure 19

III) **Translation:** Translate back to original position. So the translation matrix T2 will become

$$T2 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ Xm & Ym & 1 \end{bmatrix}$$

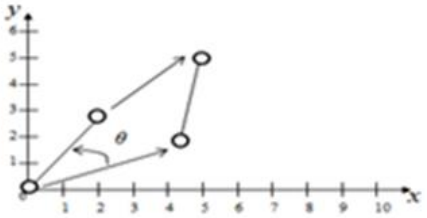


Figure 20

Now let us form a combined matrix.

M = Translation * Rotation * Translation

= T1 * R * T2

$$= \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ -Xm & -Ym & 1 \end{bmatrix} * \begin{bmatrix} Cos\theta & Sin\theta & 0 \\ -Sin\theta & Cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix} * \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ Xm & Ym & 1 \end{bmatrix}$$

$$= \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ -Xm & -Ym & 1 \end{bmatrix} * \begin{bmatrix} Cos\theta & Sin\theta & 0 \\ -Sin\theta & Cos\theta & 0 \\ Xm & Ym & 1 \end{bmatrix}$$

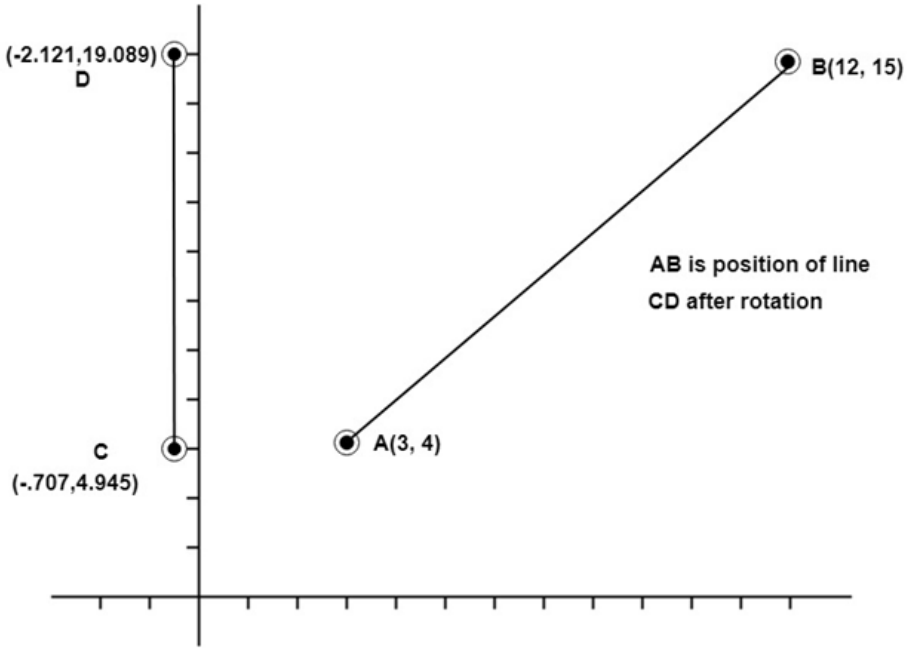
$$= \begin{bmatrix} Cos\theta & Sin\theta & 0 \\ -Sin\theta & Cos\theta & 0 \\ -Xm * Cos\theta + Ym * Sin\theta + Xm & -Xm * Sin\theta - Ym * Cos\theta + Ym & 1 \end{bmatrix}$$

Example2: Rotate a line CD whose endpoints are (3, 4) and (12, 15) about origin through a 45° anticlockwise direction.

$$\begin{matrix} \cos 45 & -\sin 45 \\ \sin 45 & \cos 45 \end{matrix} \qquad \begin{bmatrix} 0.707 & -0.707 \\ 0.707 & 0.707 \end{bmatrix}$$

$$\begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} .707 & -.707 \\ .707 & .707 \end{bmatrix} \begin{bmatrix} 3 \\ 4 \end{bmatrix} = \begin{bmatrix} .707 \times 3 - 707 \times 4 & .707 \times 3 + .707 \times 4 \end{bmatrix} = \begin{bmatrix} -.707 & 4.949 \end{bmatrix}$$

$$= \begin{bmatrix} -2.121 & 19.089 \end{bmatrix}$$



Example3: Rotate line AB whose endpoints are A (2, 5) and B (6, 12) about origin through a 30° clockwise direction.

Step1: Rotation of point A (2, 5). Take angle 30°

$$R = \begin{bmatrix} \cos 30° & -\sin 30° \\ \sin 30° & \cos 30° \end{bmatrix}$$

$$= [2, 5] \begin{bmatrix} \cos 30° & -\sin 30° \\ -\sin 30° & \cos 30° \end{bmatrix}$$

$$= [2, 5] \begin{bmatrix} .866 & -0.5 \\ .5 & .866 \end{bmatrix}$$

$$= [2 * .866 + 5 * .5 \qquad 2 * (-.5) + 5\ (.866)]$$

$$= [(1.732 + 2.5 \qquad -1 + 4.33]$$

$$= [4.232 \qquad 3.33]$$

A point (2, 5) become (4.232, 3.33)

Step2: Rotation of point B (6, 12)

$$= [6 \ \ 12] \begin{bmatrix} \cos 30° & -\sin 30° \\ \sin 30° & \cos 30° \end{bmatrix}$$

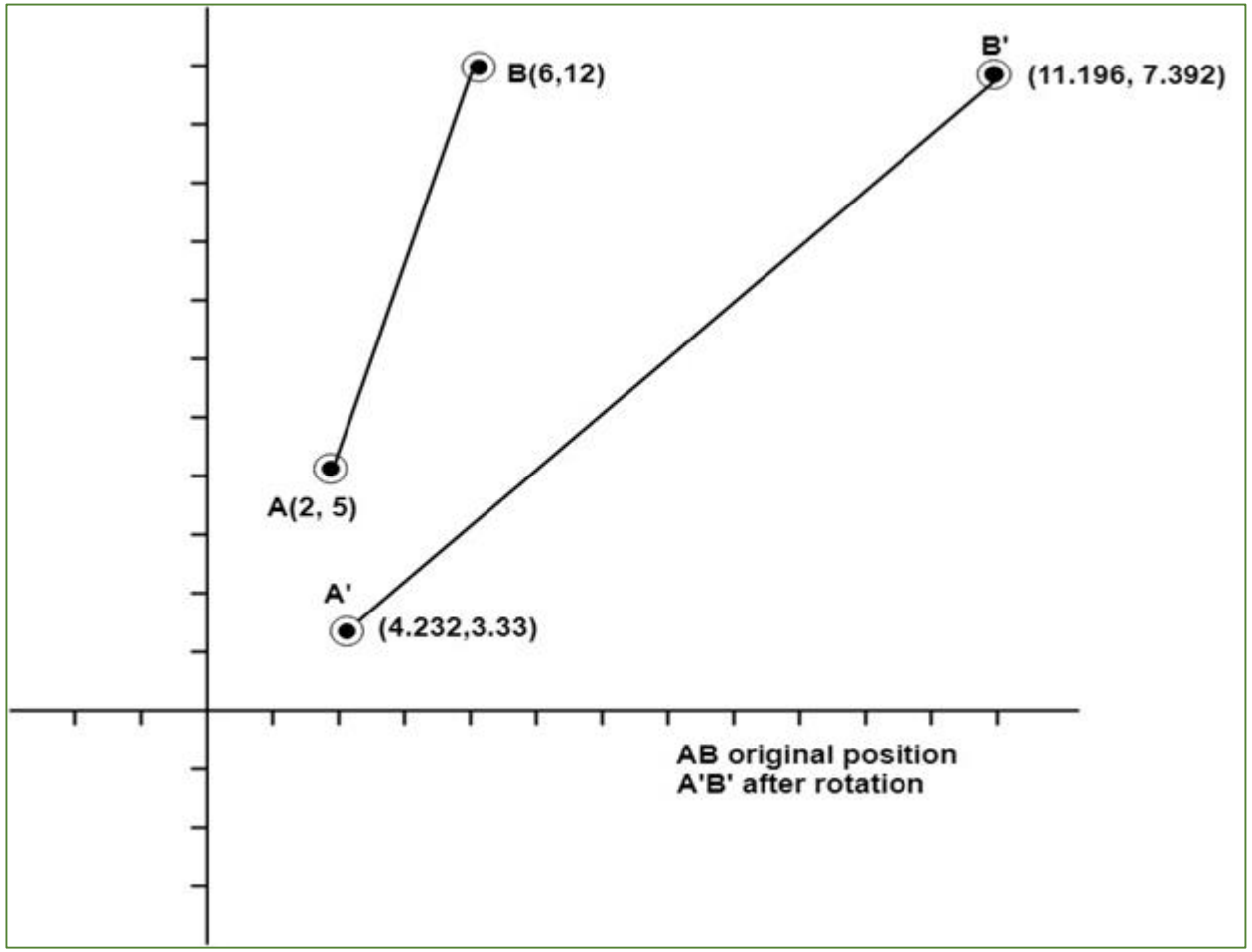
$$= [6\ 12] \begin{bmatrix} .866 & -0.5 \\ .5 & .866 \end{bmatrix}$$

$$= [6 * .866 + 12 * .5 \qquad 6 * (-.5) + 12 * (.866)]$$

$$= [5.196 + 6 \qquad -3 + 10.392]$$

$$= [11.196 \qquad 7.392]$$

B point (6, 12) becomes (11.46, 7.312) after rotation 30° in clockwise direction.



Example2: Rotate a point (12, 15) about arbitrary (3,4) through a 45° anticlockwise direction.

Class Work

❑ Scaling:

- It is used to alter or change the size of objects. The change is done using scaling factors.
- There are two scaling factors, i.e. S_x in x direction S_y in y-direction.
- If the original position is x and y. Scaling factors are S_x and S_y then the value of coordinates after scaling will be x1 and y1.
- If the picture to be enlarged to twice its original size then $S_x = S_y = 2$.
- If S_x and S_y are not equal then scaling will occur but it will elongate or distort the picture.
- If scaling factors are less than one, then the size of the object will be reduced. If scaling factors are higher than one, then the size of the object will be enlarged.
- If S_x and S_y are equal it is also called as Uniform Scaling.
- If not equal then called as Differential Scaling.
- If scaling factors with values less than one will move the object closer to coordinate origin, while a value higher than one will move coordinate position farther from origin.

$$x' = x * s_x \quad \text{and} \quad y' = y * s_y.$$

The scaling factor s_x, s_y scales the object in X and Y direction respectively. So, the above equation can be represented in matrix form:

$$\begin{bmatrix} X' \\ Y' \end{bmatrix} = \begin{bmatrix} Sx & 0 \\ 0 & Sy \end{bmatrix} \begin{bmatrix} X \\ Y \end{bmatrix} \qquad S = \begin{vmatrix} Sx & 0 & 0 \\ 0 & Sy & 0 \\ 0 & 0 & 1 \end{vmatrix}$$

Or we can say :-

$$\begin{bmatrix} X_{new} \\ Y_{new} \end{bmatrix} = \begin{bmatrix} S_x & 0 \\ 0 & S_y \end{bmatrix} \times \begin{bmatrix} X_{old} \\ Y_{old} \end{bmatrix}$$

Scaling Matrix

$$\begin{bmatrix} X_{new} \\ Y_{new} \\ 1 \end{bmatrix} = \begin{bmatrix} S_x & 0 & 0 \\ 0 & S_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \times \begin{bmatrix} X_{old} \\ Y_{old} \\ 1 \end{bmatrix}$$

Scaling Matrix

(Homogeneous Coordinates Representation)

Problem-01:

Given a square object with coordinate points A(0, 3), B(3, 3), C(3, 0), D(0, 0). Apply the scaling parameter 2 towards X axis and 3 towards Y axis and obtain the new coordinates of the object.

$$\begin{bmatrix} X' \\ Y' \end{bmatrix} = \begin{bmatrix} Sx & 0 \\ 0 & Sy \end{bmatrix} \begin{bmatrix} X \\ Y \end{bmatrix}$$

$$S = \begin{vmatrix} Sx & 0 & 0 \\ 0 & Sy & 0 \\ 0 & 0 & 1 \end{vmatrix}$$

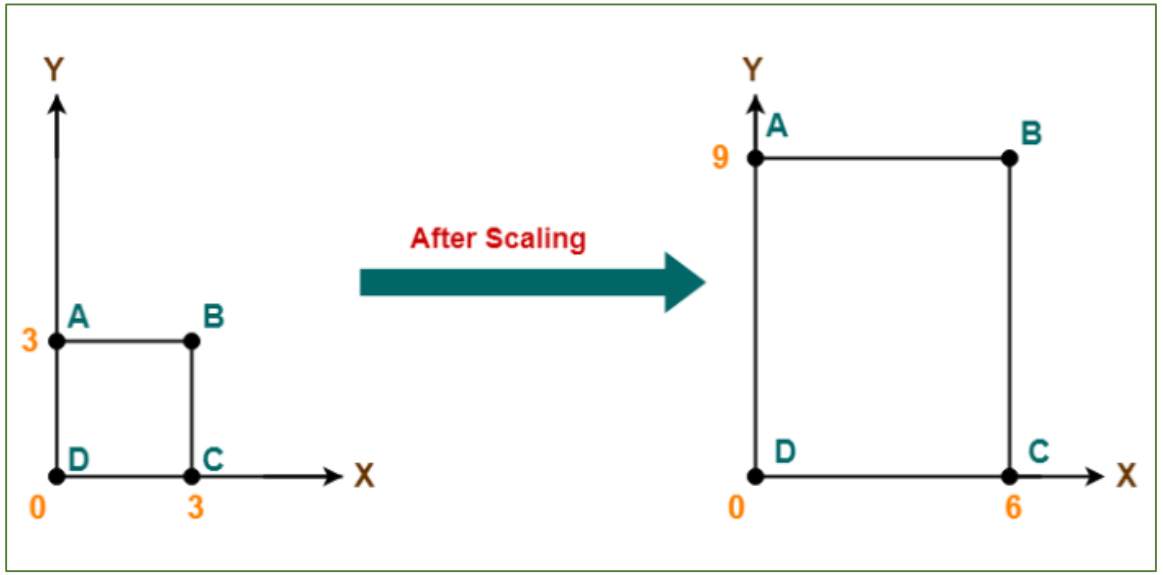
Solution:-

For point A(0,3)

We have :-

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} 2 & 0 \\ 0 & 3 \end{bmatrix} \times \begin{bmatrix} 0 \\ 3 \end{bmatrix} = \begin{bmatrix} 2 \times 0 + 0 \times 3 \\ 0 \times 0 + 3 \times 3 \end{bmatrix} = \begin{bmatrix} 0 \\ 9 \end{bmatrix}$$

Similarly for For point B,C,D we have



Problem Statement -2

Magnify a triangle placed at A(0,0), B(1,1) and C(5,2) to twice its size.

Magnification and Compression words relates to scaling of an object. So Scaling transformation should do the needful. Since we have to magnify the triangle to twice its size, we would take the scaling coordinates Sx and Sy as 2. Sx =2 and Sy =2.

Please remember if nothing is mentioned about the point of transformation then assume it from origin.

So New Triangle Coordinates = Scaling Matrix * Triangle Matrix.

$$= \begin{bmatrix} Sx & 0 & 0 \\ 0 & Sy & 0 \\ 0 & 0 & 1 \end{bmatrix} \times \begin{array}{c} \begin{array}{ccc} A & B & C \end{array} \\ \begin{bmatrix} 0 & 1 & 5 \\ 0 & 1 & 2 \\ 1 & 1 & 1 \end{bmatrix} \end{array} = \begin{bmatrix} 2 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 1 \end{bmatrix} \times \begin{array}{c} \begin{array}{ccc} A & B & C \end{array} \\ \begin{bmatrix} 0 & 1 & 5 \\ 0 & 1 & 2 \\ 1 & 1 & 1 \end{bmatrix} \end{array}$$

$$= \begin{array}{c} \begin{array}{ccc} A' & B' & C' \end{array} \\ \begin{bmatrix} 0 & 2 & 10 \\ 0 & 2 & 4 \\ 1 & 1 & 1 \end{bmatrix} \end{array}$$

Problem Statement-1

Magnify a triangle placed at A (0,0), B (1,1) and C (5,2) to twice its size keeping the point C (5,2) Fixed.

If the point C (5,2) needs to be fixed this means the transformation scaling needs to be done with respect to the point C (5,2). So, we will bring the point C (5,2) to origin first and then perform the transformation and finally send the point C (5,2) back.

In other words, we would perform a negative translation with Tx =5 and Ty = 2 then perform scaling transformation Sx and Sy =2 and finally a positive translation with Tx=5 and Ty=2.

So New Triangle Coordinates = **Positive Translation Matrix * Scaling Matrix * Negative Translation Matrix**

$$= \begin{bmatrix} 1 & 0 & tx \\ 0 & 1 & ty \\ 0 & 0 & 1 \end{bmatrix} * \begin{bmatrix} sx & 0 & 0 \\ 0 & sy & 0 \\ 0 & 0 & 1 \end{bmatrix} * \begin{bmatrix} 1 & 0 & -tx \\ 0 & 1 & -ty \\ 0 & 0 & 1 \end{bmatrix}$$

The Composite Scaling Transformation

With values tx=5, ty=2 and sx and sy=2.

$$= \begin{bmatrix} 1 & 0 & 5 \\ 0 & 1 & 2 \\ 0 & 0 & 1 \end{bmatrix} * \begin{bmatrix} 2 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 1 \end{bmatrix} * \begin{bmatrix} 1 & 0 & -5 \\ 0 & 1 & -2 \\ 0 & 0 & 1 \end{bmatrix}$$

With Values of tx, ty,sx and sy

$$= \begin{bmatrix} 2 & 0 & 5 \\ 0 & 2 & 2 \\ 0 & 0 & 1 \end{bmatrix} * \begin{bmatrix} 1 & 0 & -5 \\ 0 & 1 & -2 \\ 0 & 0 & 1 \end{bmatrix}$$

$$= \begin{bmatrix} 2 & 0 & -5 \\ 0 & 2 & -2 \\ 0 & 0 & 1 \end{bmatrix}$$

$$= \begin{bmatrix} 2 & 0 & -5 \\ 0 & 2 & -2 \\ 0 & 0 & 1 \end{bmatrix} \begin{array}{c} \begin{array}{ccc} A & B & C \end{array} \\ \begin{bmatrix} 0 & 1 & 5 \\ 0 & 1 & 2 \\ 1 & 1 & 1 \end{bmatrix} \end{array} = \begin{array}{c} \begin{array}{ccc} A' & B' & C' \end{array} \\ \begin{bmatrix} -5 & -3 & 5 \\ -2 & 0 & 2 \\ 1 & 1 & 1 \end{bmatrix} \end{array}$$

Problem Statement-1

Magnify a triangle placed at A (0,0), B (1,1) and C (5,2) to twice its size keeping the point C (5,2) Fixed.

$$\begin{array}{c} \begin{array}{ccc} A & B & C \end{array} \\ \begin{bmatrix} 0 & 1 & 5 \\ 0 & 1 & 2 \\ 1 & 1 & 1 \end{bmatrix} \end{array}$$

The new triangle coordinates are A'(-5,-2), B(-3,0) and C'(5,2). You can see point C(5,2) remained unchanged after scaling transformation. Which confirms that the solution is correct.

❑ Reflection:

It is a transformation which produces a **mirror image of an objec**t. The mirror image can be either about x-axis or y-axis. The object is rotated by180°.

a. Types of Reflection:

b. Reflection about the x-axis

c. Reflection about the y-axis

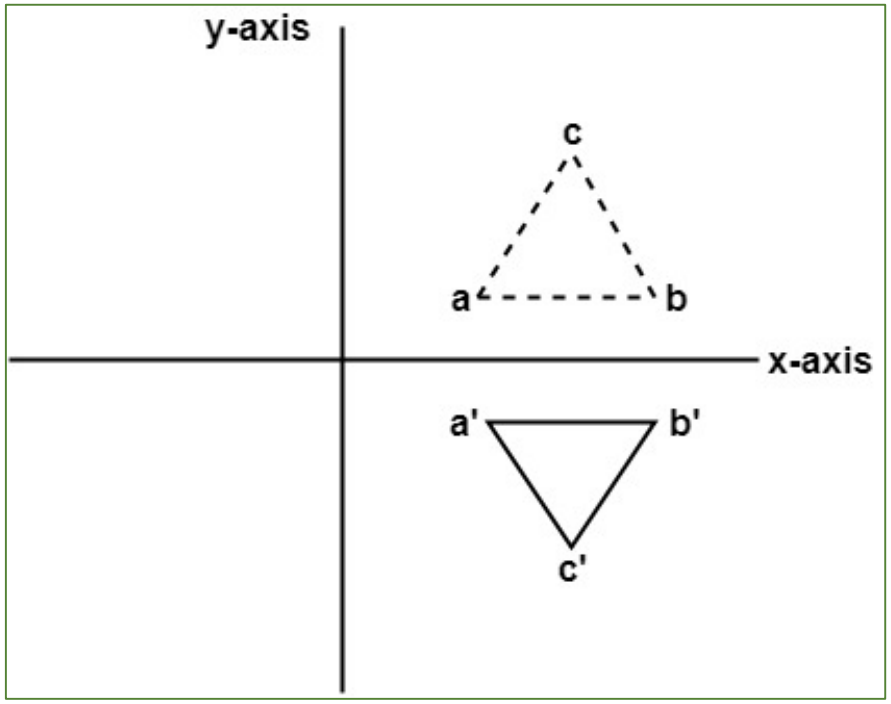
d. Reflection about an axis perpendicular to xy plane and passing through the origin

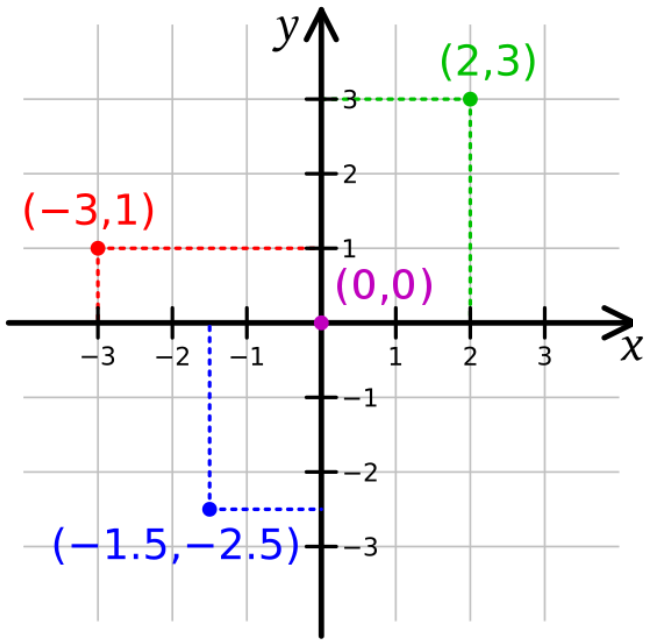
e. Reflection about line y=x

a. Reflection about x-axis: The object can be reflected about x-axis with the help of the following matrix

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

- In this transformation value of x will remain same whereas the value of y will become negative. Following figures shows the reflection of the object axis. The object will lie another side of the x-axis.





$x' = x$

$y' = -y$

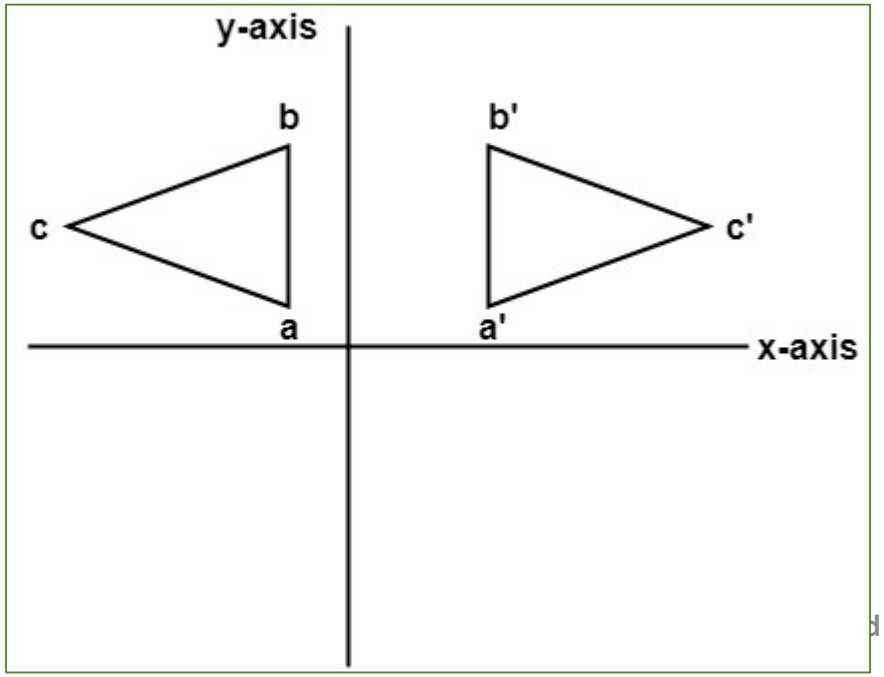
$$\begin{pmatrix} x' \\ y' \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix}$$

2. Reflection about y-axis: The object can be reflected about y-axis with the help of following transformation matrix

$$\begin{bmatrix} -1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

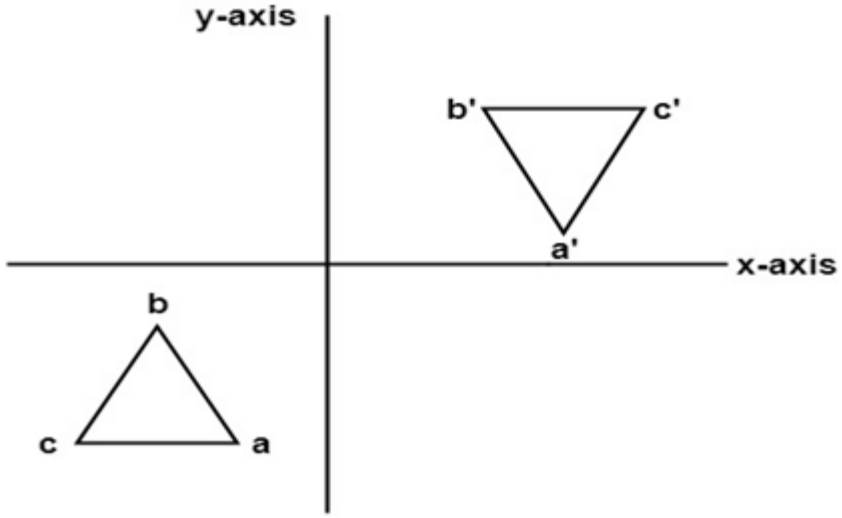
Here the values of x will be reversed, whereas the value of y will remain the same. The object will lie another side of the y-axis.

The following figure shows the reflection about the y-axis



3. Reflection about an axis perpendicular to xy plane and passing through origin:
In the matrix of this transformation is given below

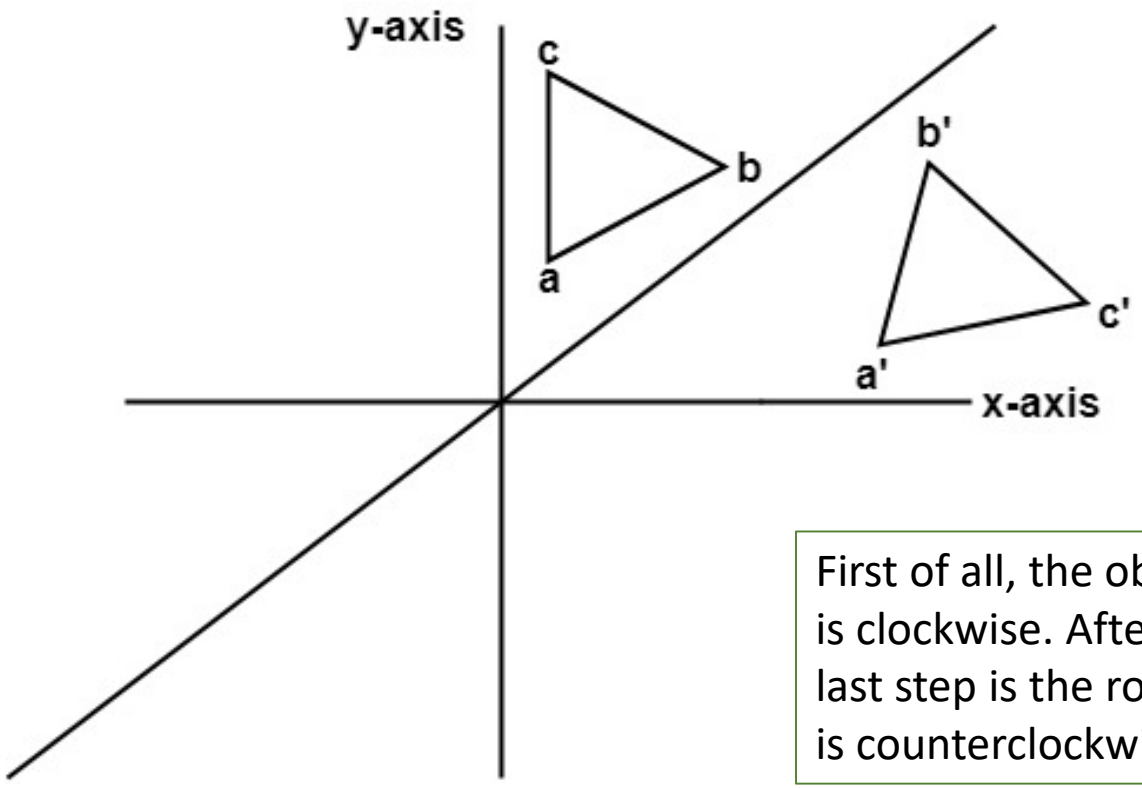
$$\begin{bmatrix} -1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$



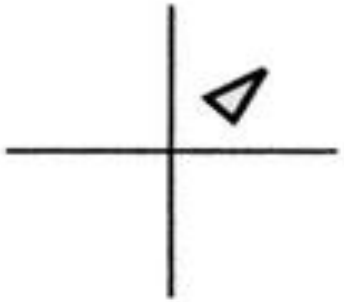
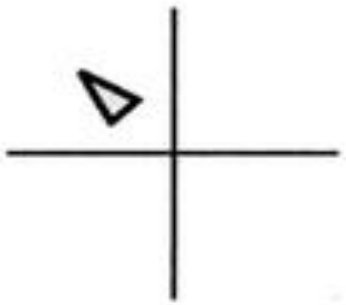
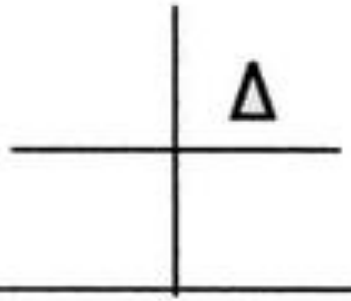
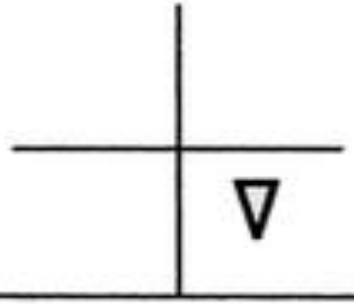
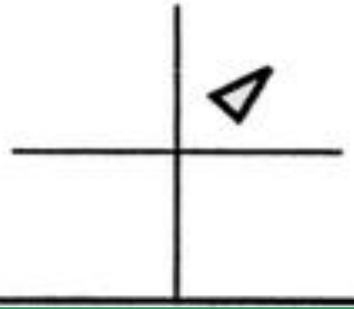
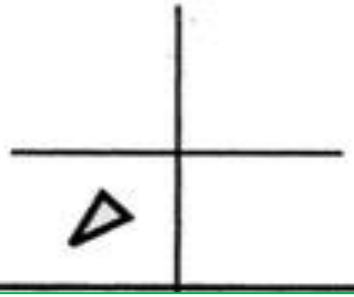
In this value of x and y both will be reversed. This is also called as half revolution about the origin.

4. Reflection about line y=x: The object may be reflected about line y = x with the help of following transformation matrix

$$\begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

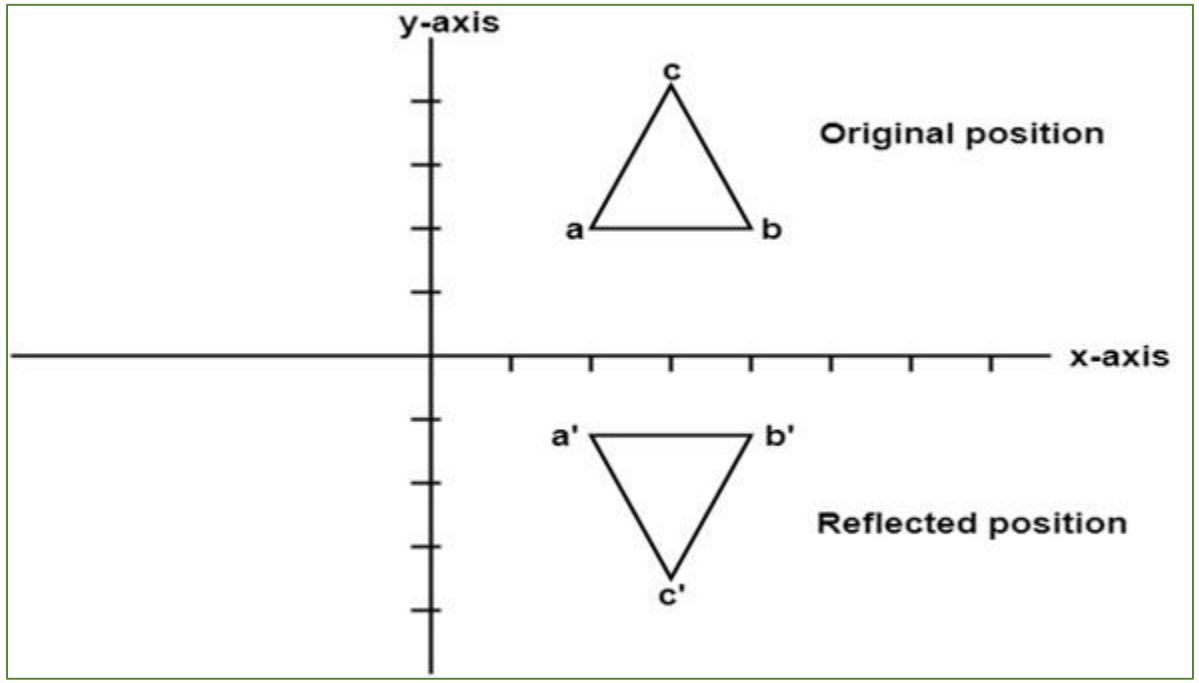


First of all, the object is rotated at 45°. The direction of rotation is clockwise. After it reflection is done concerning x-axis. The last step is the rotation of y=x back to its original position that is counterclockwise at 45°.

Reflection	Transformation matrix	Original image	Reflected image
Reflection about Y axis	$\begin{bmatrix} -1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$		
Reflection about x axis	$\begin{bmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$		
Reflection about origin	$\begin{bmatrix} -1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$		

Example: A triangle ABC is given. The coordinates of A, B, C are given as

A (3 4)
B (6 4)
C (4 8)



The matrix for reflection about x axis $\begin{bmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$

The a point coordinates after reflection

$$(x, y) = (3, 4) \begin{bmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

(x, y) = [3, -4] Explain in class how multiply

$$(x, y) = (6, 4) \begin{bmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

(x, y) = [6, -4]

$$(x, y) = (4, 8) \begin{bmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

(x, y) = [4, -8]

a (3, 4) becomes a1 (3, -4)
b (6, 4) becomes b1 (6, -4)
c (4, 8) becomes c1 (4, -8)

Some Important Questions

1. A point X has coordinates (-1, -8). The point is reflected across the y-axis to form X'. X' is translated over 4 to the right and up 6 to form X''. What are the coordinates of X' and X''?

2. A point A has coordinates (2, -3). The point is translated over 3 to the left and up 5 to form A'. A' is reflected across the x-axis to form A''. What are the coordinates of A' and A''?

3. A point P has coordinates (5, -6). The point is reflected across the line $y = -x$ to form P'. P' is rotated about the origin $90°$ CW to form P''. What are the coordinates of P' and P''?

4. Line JT has coordinates $J(-2,-5)$ and $T(2,3)$. The segment is rotated about the origin $180°$ to form $J'T'$. $J'T'$ is translated over 6 to the right and down 3 to form $J''T''$. What are the coordinates of $J'T'$ and $J''T''$?

5. Line SK has coordinates $S(-1,-8)$ and $K(1,2)$. The segment is translated over 3 to the right and up 3 to form $S'K'$. $S'K'$ is rotated about the origin $90°$ CCW to form $S''K''$. What are the coordinates of $S'K'$ and $S''K''$?

6. A point K has coordinates (-1, 4). The point is reflected across the line $y = x$ to form K'. K' is rotated about the origin $270°$ CW to form K''. What are the coordinates of K' and K''?

❑ Shearing

- ❑ It deals with changing the shape and size of the 2D object along x-axis and y-axis.
- ❑ It is similar to sliding the layers in one direction to change the shape of the 2D object.
- ❑ It is an ideal technique to change the shape of an existing object in a two dimensional plane.
- ❑ In a two dimensional plane, the object size can be changed along X direction as well as Y direction.

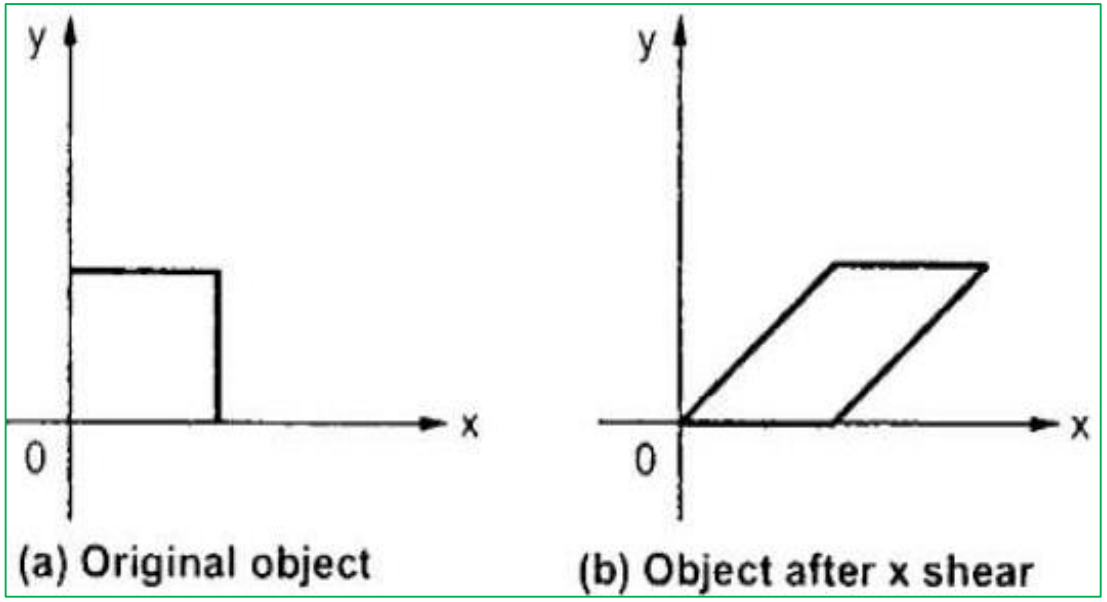
x-Shear :

In x shear, the y co-ordinates remain the same but the x co-ordinates changes.

I f P(x, y) is the point then the new points will be P'(x', y') given as –

$$x' = x + Sh_x * y; y' = y$$

Matrix Form:



(a) Original object (b) Object after x shear

In 'x' direction,

$x' = x + s_{hx} \cdot y$

$y' = y$

$$\begin{pmatrix} x' \\ y' \end{pmatrix} = \begin{pmatrix} 1 & s_{hx} \\ 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix}$$

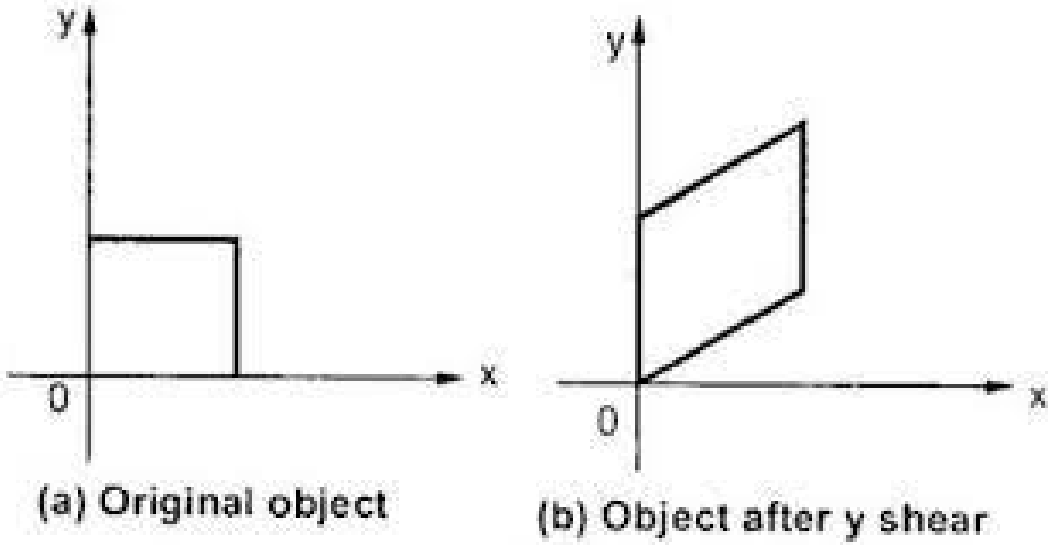
OR We Can Say in other Matrix Form:

$$\begin{bmatrix} x' & y' \end{bmatrix} = \begin{bmatrix} x & y \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 \\ Sh_x & 1 \end{bmatrix}$$

Homogeneous Coordinate Representation

$$X_{sh} = \begin{bmatrix} 1 & shx & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Shearing in the Y-direction: Here shearing is done by sliding along vertical or y-axis.



(a) Original object (b) Object after y shear

In 'y' direction,

$x' = x$

$y' = y + S_{hy} \cdot x$

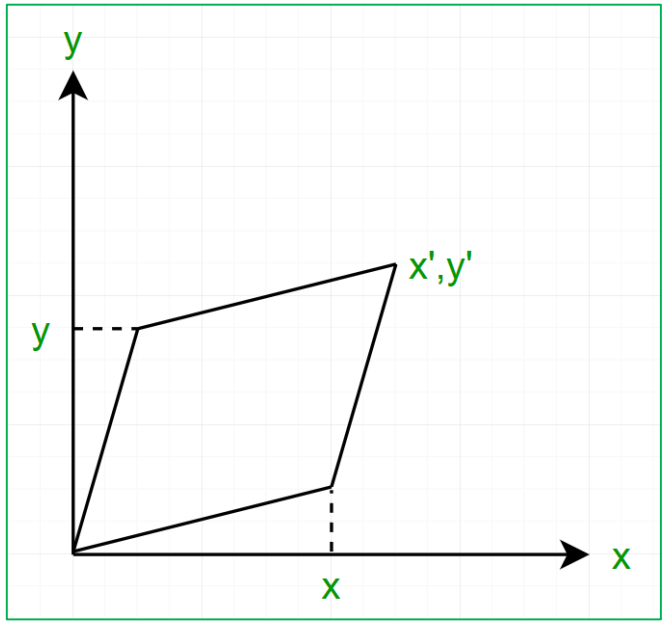
$$\begin{pmatrix} x' \\ y' \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ S_{hy} & 1 \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix}$$

OR We Can Say in other Matrix Form:

$$[x' \quad y'] = [x \quad y] \cdot \begin{bmatrix} 1 & Sh_y \\ 0 & 1 \end{bmatrix}$$

Homogeneous Coordinate Representation

$$Y_{sh} \begin{bmatrix} 1 & 0 & 0 \\ shy & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$



In both directions,

$$x' = x + s_{hx} \cdot y$$
$$y' = y + s_{hy} \cdot x$$

$$\begin{pmatrix} x' \\ y' \end{pmatrix} = \begin{pmatrix} 1 & s_{hx} \\ s_{hy} & 1 \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix}$$

Homogeneous Coordinate Representation

Example :
Given a triangle with points (1, 1), (0, 0) and (1, 0).
Find out the new coordinates of the object along x-axis, y-axis, xy-axis.
(Applying shear parameter 4 on X-axis and 1 on Y-axis.).

Given,
Old corner coordinates of the triangle = A (1, 1), B(0, 0), C(1, 0)
Shearing parameter along X-axis (Shx) = 4
Shearing parameter along Y-axis (Shy) = 1

Along x-axis:
A'=(1+4*1, 1)=(5, 1)
B'=(0+4*0, 0)=(0, 0)
C'=(1+4*0, 0)=(1, 0)

Along y-axis:
A''=(1, 1+1*1)=(1, 2)
B''=(0, 0+1*0)=(0, 0)
C''=(1, 0+1*1)=(1, 1)

Along xy-axis:
A'''=(1+4*1, 1+1*1)=(5, 2)
B'''=(0+4*0, 0+1*0)=(0, 0)
C'''=(1+4*0, 0+1*1)=(1, 1)

Solve it by Matrix representation :-Class work

Row Method

1. Translation

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ t_x & t_y & 1 \end{bmatrix}$$

2. Scaling

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ S_x & S_y & 1 \end{bmatrix}$$

Column Method

1.Translation

$$\begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix}$$

2. Scaling

$$\begin{bmatrix} 1 & 0 & S_x \\ 0 & 1 & S_y \\ 0 & 0 & 1 \end{bmatrix}$$

Revised Row method and column method of Matrix representation

Assignment Work

Q.N.1. Find the coordinate of a triangle A(1,3) , B(2,5) and C(3,3) after twice its original size

a) about origin

b)about fixed point (2,3)

Q.N.2. Find the coordinate of a triangle A(1,3) , B(2,5) and C(3,3) after being rotated about fixed point p(2,4) by 45 in clockwise direction and then translate (3,4) .

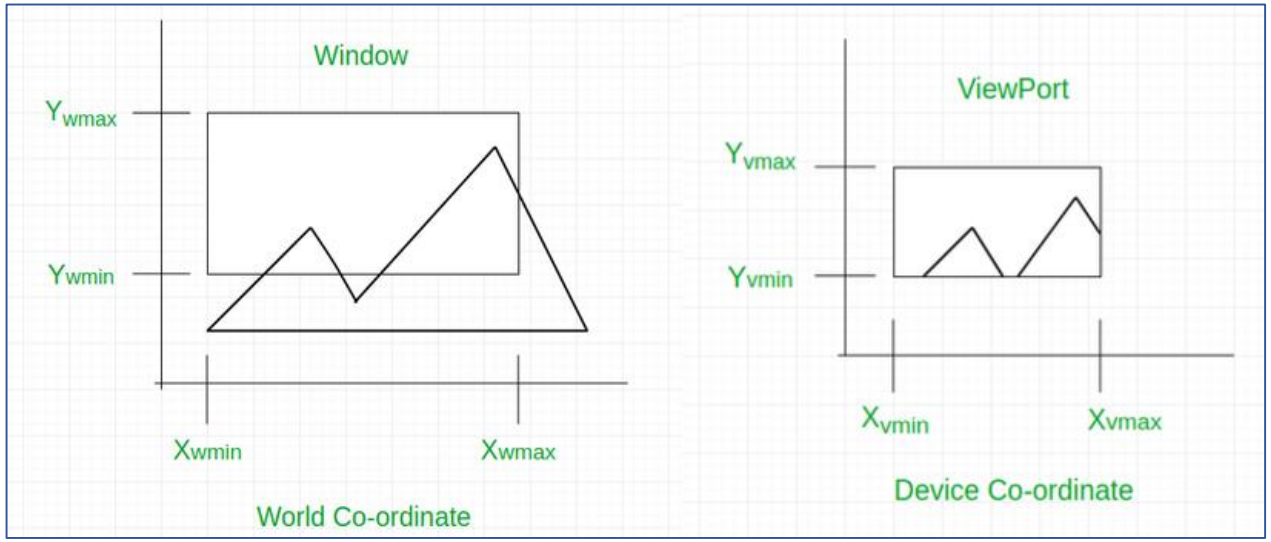
Q.N.3.Find the coordinate of a triangle A(1,3) , B(2,5) and C(3,3) after being rotated about fixed point p(2,4) by 45 in clockwise direction and then translate by 3 unit along x-direction.

Q.N.4 Find the coordinate of a triangle (5, 5), (7, 3), (3, 3) after scaling (2,3) and then rotate 45 CCW then translate (1,1) and then reflect y=0 line.

Q.N.5. Find the coordinate of a line (5, 5), (3, 3) after rotate 450 ccw at fixed point (3,3) then translate (1,1) then reflect at x-axis.

3.3 Two Dimensional Viewing: Viewing pipeline, Window to viewport coordinate transformation

- **Window to Viewport Transformation** is the process of transforming 2D world-coordinate objects to device coordinates. Objects inside the world or clipping window are mapped to the viewport which is the area on the screen where world coordinates are mapped to be displayed.



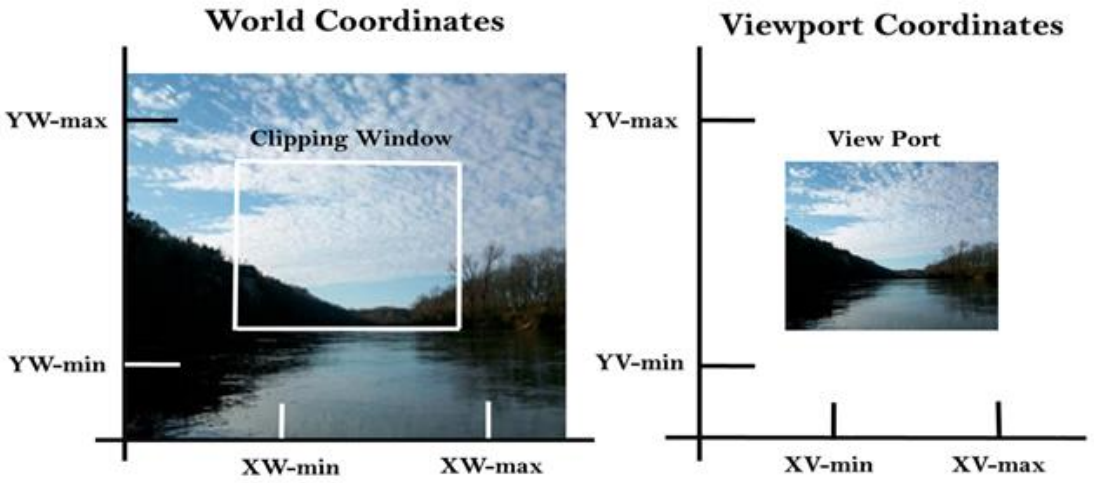


Fig: Window to viewport mapping

General Terms:

- **World coordinate –** It is the Cartesian coordinate w.r.t which we define the diagram, like X_{wmin}, X_{wmax}, Y_{wmin}, Y_{wmax}
- **Device Coordinate –**It is the screen coordinate where the objects are to be displayed, like X_{vmin}, X_{vmax}, Y_{vmin}, Y_{vmax}
- **Window –**It is the area on the world coordinate selected for display.
- **ViewPort –**It is the area on the device coordinate where graphics is to be displayed.

Mathematical Calculation of Window to Viewport:

It may be possible that the size of the Viewport is much smaller or greater than the Window.

In these cases, we have to increase or decrease the size of the Window according to the Viewport and for this, we need some mathematical calculations.

(x_w, y_w): A point on Window
(x_v, y_v): Corresponding point on Viewport

- In order to maintain the same relative placement of the point in the viewport as in the window, we require

$$\frac{xv - xv_{min}}{xv_{max} - xv_{min}} = \frac{xw - xw_{min}}{xw_{max} - xw_{min}} \quad \text{.....equation 1}$$

$$\frac{yv - yv_{min}}{yv_{max} - yv_{min}} = \frac{yw - yw_{min}}{yw_{max} - yw_{min}}$$

:

- Solving these impressions for the viewport position (xv, yv), we have

$$xv = xv_{min} + (xw - xw_{min})sx$$

$$yv = yv_{min} + (yw - yw_{min})sy \quad \text{.....equation 2}$$

Where scaling factors are

$$sx = \frac{xv_{max} - xv_{min}}{xw_{max} - xw_{min}}$$

$$sy = \frac{yv_{max} - yv_{min}}{yw_{max} - yw_{min}}$$

- Above Equation (1) and Equation (2) can also be derived with a set of transformation that converts the window or world coordinate area into the viewport or screen coordinate area.
- This conversation is performed with the following sequence of transformations:
- Perform a scaling transformation using a fixed point position (xw_{min},yw) that scales the window area to the size of the viewport. $_{min}$
- Translate the scaled window area to the position of the viewport. Relative proportions of objects are maintained if the scaling factors are the same (sx=sy).
- From normalized coordinates, object descriptions are mapped to the various display devices.

- Any number of output devices can we open in a particular app, and three windows to viewport transformation can be performed for each open output device.

- This mapping called workstation transformation (It is accomplished by selecting a window area in normalized space and a viewport area in the coordinates of the display device).

- As in fig, workstation transformation to partition a view so that different parts of normalized space can be displayed on various output devices).

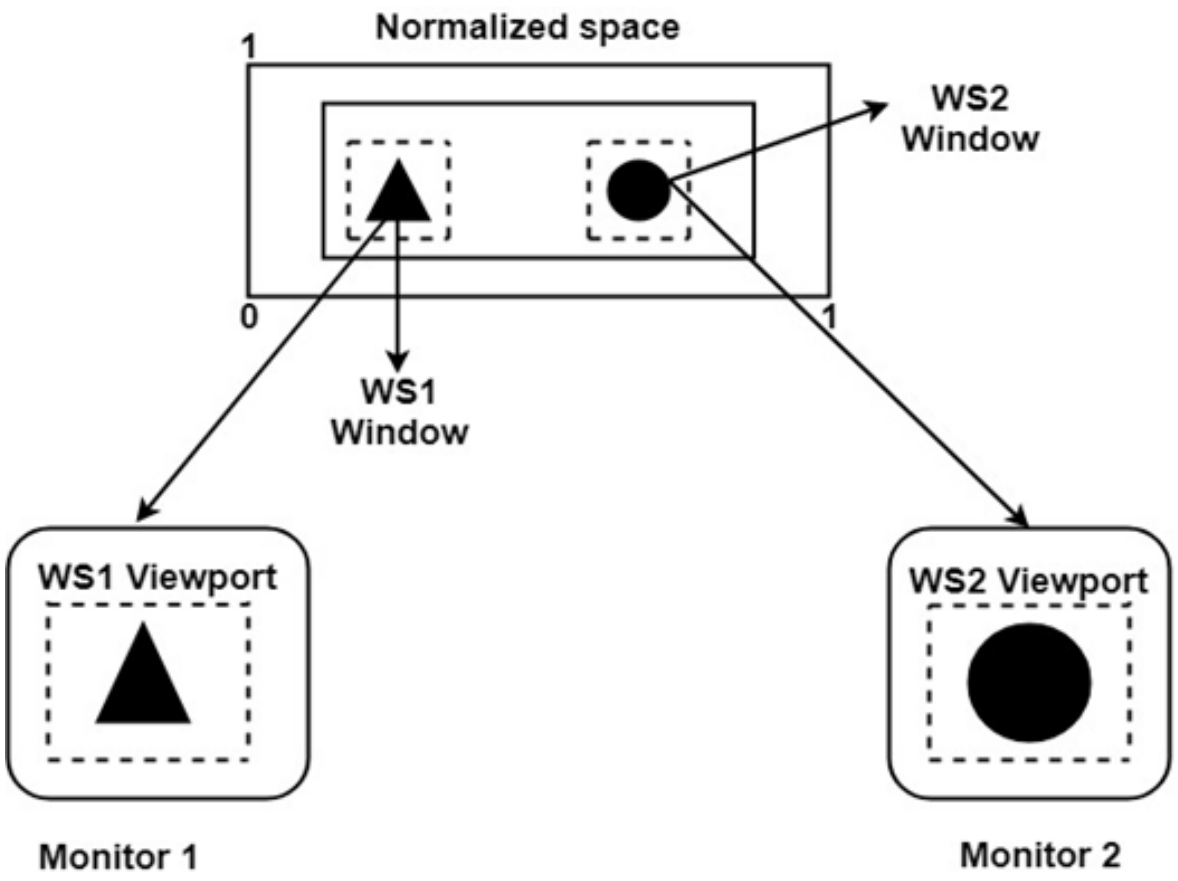
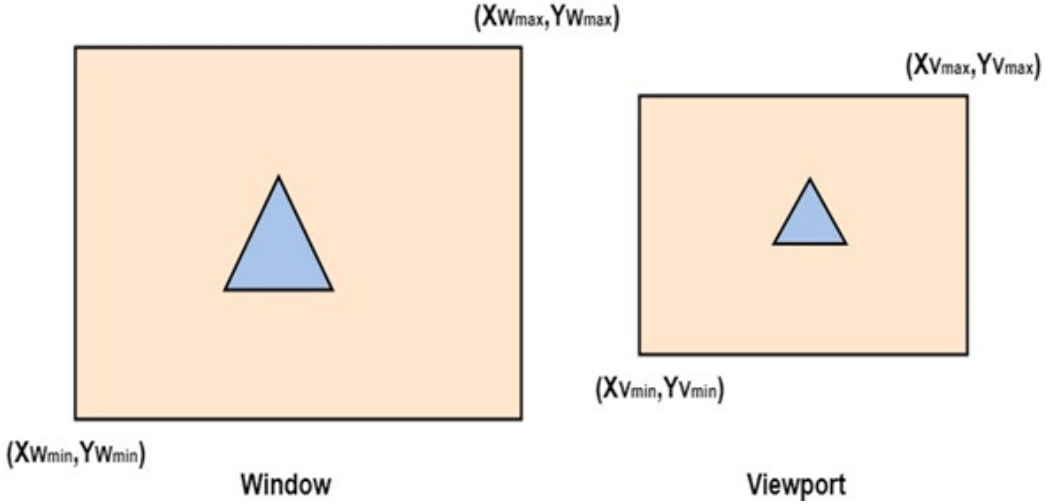


Fig:Mapping selected parts of a scene in normalized coordinates to different video monitors with workstation transformation.

❑ Matrix Representation of the above three steps of Transformation:



Step1: Translate window to origin 1

$T_x = -Xw_{min}$ $T_y = -Yw_{min}$

Step2: Scaling of the window to match its size to the viewport

$S_x = (Xy_{max} - Xv_{min})/(Xw_{max} - Xw_{min})$

$S_y = (Yv_{max} - Yv_{min})/(Yw_{max} - Yw_{min})$

Step3: Again translate viewport to its correct position on screen.

$T_x = Xv_{min}$

$T_y = Yv_{min}$

Prepared By:-P.B.Jha

$$T = \begin{vmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ -Xw_{min} & -Yw_{min} & 1 \end{vmatrix}$$

$$S = \begin{vmatrix} S_x & 0 & 0 \\ 0 & S_y & 0 \\ 0 & 0 & 1 \end{vmatrix}$$

$$T_1 = \begin{vmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ Xv_{min} & Yv_{min} & 1 \end{vmatrix}$$

Viewing Transformation= T * S * T1

Example: 1

- for window, $\mathbf{X_{wmin} = 20, X_{wmax} = 80, Y_{wmin} = 40, Y_{wmax} = 80}$.
- for viewport, $\mathbf{X_{vmin} = 30, X_{vmax} = 60, Y_{vmin} = 40, Y_{vmax} = 60}$.
- Now a point (X_w, Y_w) be (30, 80) on the window. We have to calculate that point on the viewport

  i.e **(X_v, Y_v)**.
- First of all, calculate the scaling factor of x coordinate S_x and the scaling factor of y coordinate S_y using the above-mentioned formula.

```
Sx = ( 60 - 30 ) / ( 80 - 20 ) = 30 / 60
Sy = ( 60 - 40 ) / ( 80 - 40 ) = 20 / 40
```

$$sx = \frac{xv_{max} - xv_{min}}{xw_{max} - xw_{min}}$$

$$sy = \frac{yv_{max} - yv_{min}}{yw_{max} - yw_{min}}$$

- So, now calculate the point on the viewport (X_v, Y_v).

```
Xv = 30 + ( 30 - 20 ) * ( 30 / 60 ) = 35
Yv = 40 + ( 80 - 40 ) * ( 20 / 40 ) = 60
```

$$xv = xv_{min} + (xw - xw_{min})sx$$

$$yv = yv_{min} + (yw - yw_{min})sy \text{ .....equation 2}$$

- So, the point on window (X_w, Y_w) = (30, 80) will be **(X_v, Y_v) = (35, 60)** on viewport.

- Advantage of Viewing Transformation:
- We can display picture at device or display system according to our need and choice.

Note:

- World coordinate system is selected suits according to the application program.
- Screen coordinate system is chosen according to the need of design.
- Viewing transformation is selected as a bridge between the world and screen coordinate.
-

Example -2 Solve by Matrix Method

for window, $Xw_{min} = 20$, $Xw_{max} = 80$, $Yw_{min} = 40$, $Yw_{max} = 80$.

for viewport, $Xv_{min} = 30$, $Xv_{max} = 60$, $Yv_{min} = 40$, $Yv_{max} = 60$.

Now a point (X_w, Y_w) be (30, 80) on the window. We have to calculate that point on the viewport i.e (X_v, Y_v).

$$T = \begin{vmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ -Xw_{min} & -Yw_{min} & 1 \end{vmatrix}$$

$$S = \begin{vmatrix} S_x & 0 & 0 \\ 0 & S_y & 0 \\ 0 & 0 & 1 \end{vmatrix}$$

$$T_1 = \begin{vmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ Xv_{min} & Yv_{min} & 1 \end{vmatrix}$$

❑ Clipping:

Clipping in computer graphics refers to removing objects or parts of objects outside of the visible area of a computer screen or viewport.

2. Why Do We Need Clipping?

The viewing window (or viewport) often has a limited display area. Moreover, when rendering complex scenes, it's essential to optimize performance by discarding objects or parts of objects that viewers can't see.

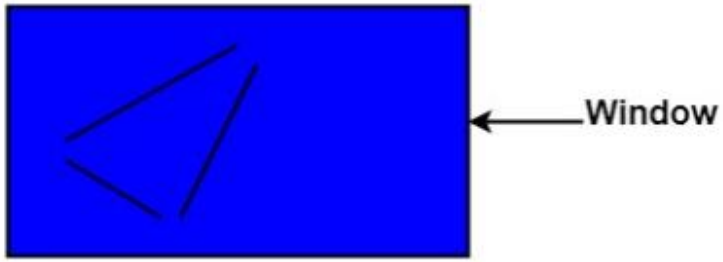
Clipping helps eliminate unnecessary calculations and improves the efficiency of rendering algorithms. Furthermore, it guarantees the display of only the visible portions of objects, resulting in faster rendering times and a more realistic representation of a scene.

❑ Types of Lines:

Lines are of three types:

a. Visible: A line or lines entirely inside the window is considered visible
b. Invisible: A line entirely outside the window is considered invisible
c. Clipped: A line partially inside the window and partially outside is clipped. For clipping point of intersection of a line with the window is determined.

Case1:

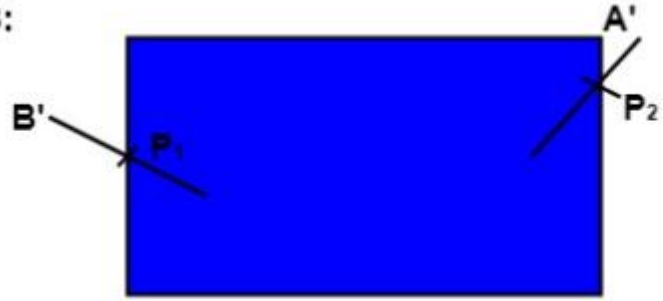


Visible Lines

Case2:

Invisible Lines

Case3:



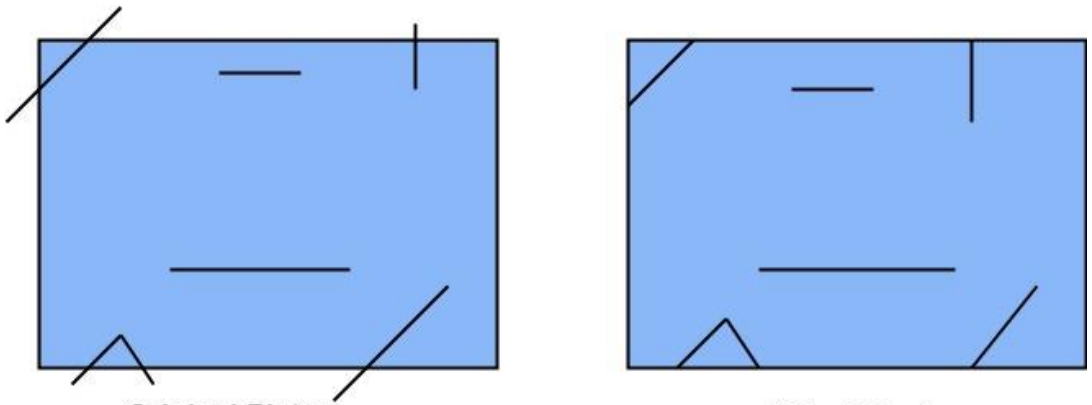
Clipped Lines

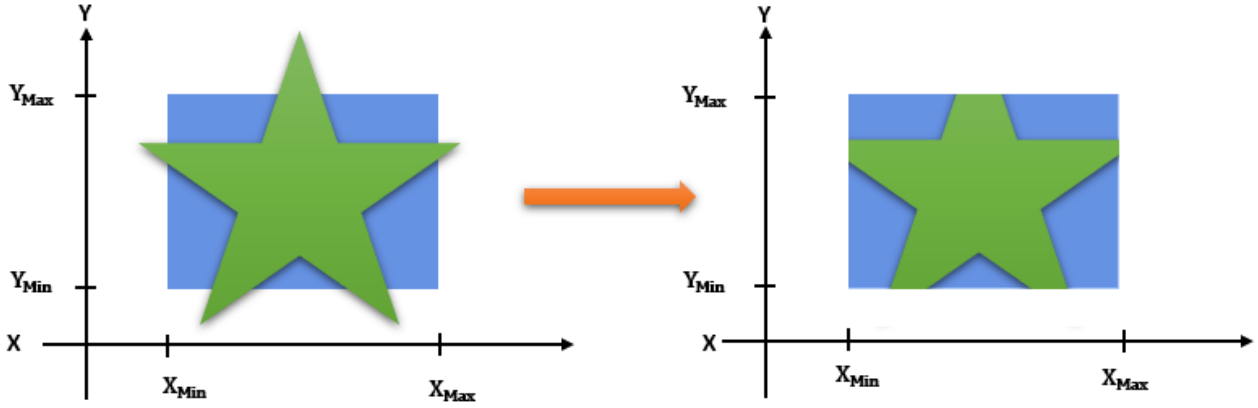
In this figure P_1 and P_2 are point of intersection. The line P_2 to A' and P_1 to B' is discarded or clipped.

Clipping can be applied through hardware as well as software. In some computers, hardware devices automatically do work of clipping. In a system where hardware clipping is not available software clipping applied.

Following figure show before and after clipping

The window against which object is clipped called a clip window. It can be curved or rectangle in shape.





Before Clipping

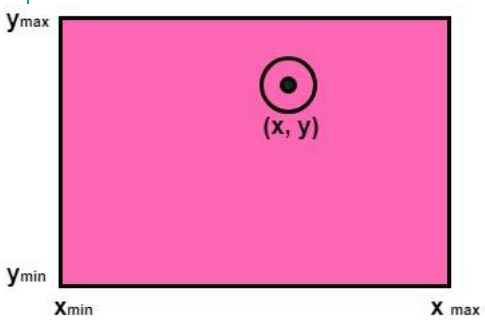
After Clipping

❑ Applications of clipping:

- It will extract part we desire.
- For identifying the visible and invisible area in the 3D object.
- For creating objects using solid modeling.
- For drawing operations.
- Operations related to the pointing of an object.
- For deleting, copying, moving part of an object.
- Clipping can be applied to world co-ordinates. The contents inside the window will be mapped to device co-ordinates. Another alternative is a complete world co-ordinates picture is assigned to device co-ordinates, and then clipping of viewport boundaries is done.

❑ Types of Clipping:

a. Point Clipping
b. Line Clipping
c. Area Clipping (Polygon)
d. Curve Clipping
e. Text Clipping
f. Exterior Clipping



❑ Point Clipping:

Point Clipping is used to determining, whether the point is inside the window or not. For this following conditions are checked.

1. $x \leq xmax$
2. $x \geq xmin$
3. $y \leq ymax$
4. $y \geq ymin$

The (x, y) is coordinate of the point. If anyone from the above inequalities is false, then the point will fall outside the window and will not be considered to be visible.

Algorithm of Point Clipping:

Step 1: First, we set the value of xwmin and xwmax coordinates for the window.

Step 2: Now, set the coordinates of a given point (P, Q).

Step 3: Nowcheck the above mention condition.

Step 4: If

Point coordinates lie between the (xwmin, xwmax) and (ywmin, ywmax)

Then

{Display the point in the view pane}

Else

{Removethe point}

Step 5: Stop.

❑ Line Clipping

The concept of line clipping is same as point clipping. In line clipping, we will cut the portion of line which is outside of window and keep only the portion that is inside the window.

It is performed by using the line clipping algorithm. The line clipping algorithms are:

1. Cohen Sutherland Line Clipping Algorithm
2. Midpoint Subdivision Line Clipping Algorithm
3. Liang-Barsky Line Clipping Algorithm

❑ Cohen Sutherland Line Clipping Algorithm:

In the algorithm, first of all, it is detected whether line lies inside the screen or it is outside the screen. All lines come under any one of the following categories:

1. Visible
2. Not Visible
3. Clipping Case

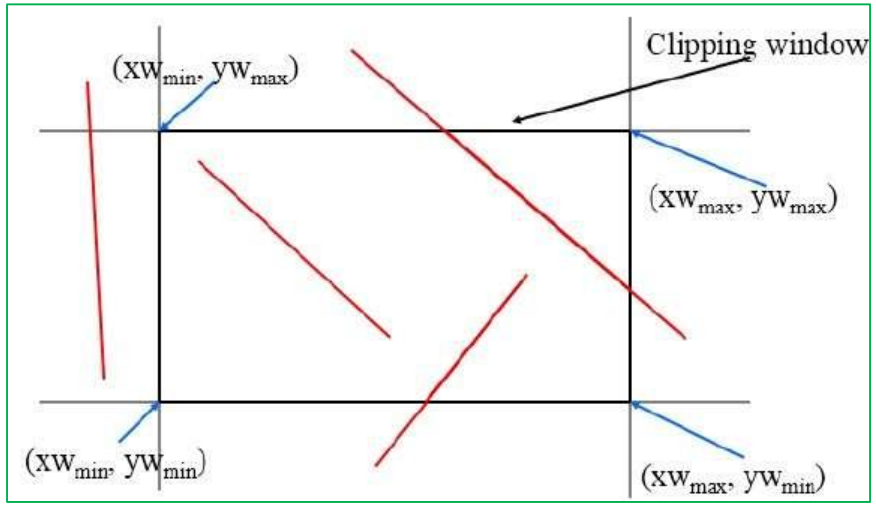
1. Visible: If a line lies within the window, i.e., both endpoints of the line lies within the window. A line is visible and will be displayed as it is.

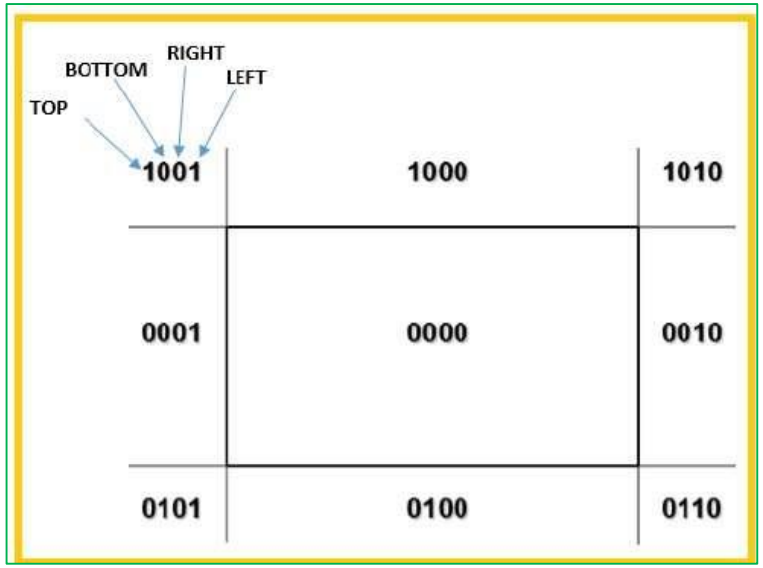
2. Not Visible: If a line lies outside the window it will be invisible and rejected. Such lines will not display. If any one of the following inequalities is satisfied, then the line is considered invisible. Let A (x1,y2) and B (x2,y2) are endpoints of line.

x_{min},x_{max} are coordinates of the window.

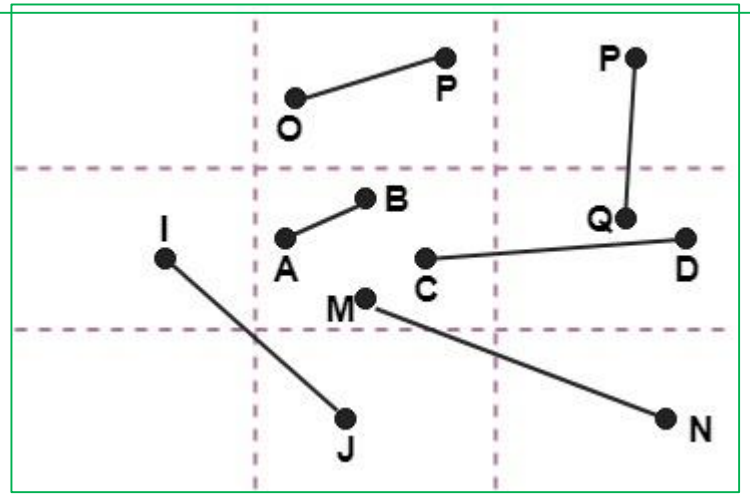
y_{min},y_{max} are also coordinates of the window.

$x1>x_{max}$
$x2>x_{max}$
$y1>y_{max}$
$y2>y_{max}$
$x1<x_{min}$
$x2<x_{min}$
$y1<y_{min}$
$y2<y_{min}$





Following figure show lines of various types



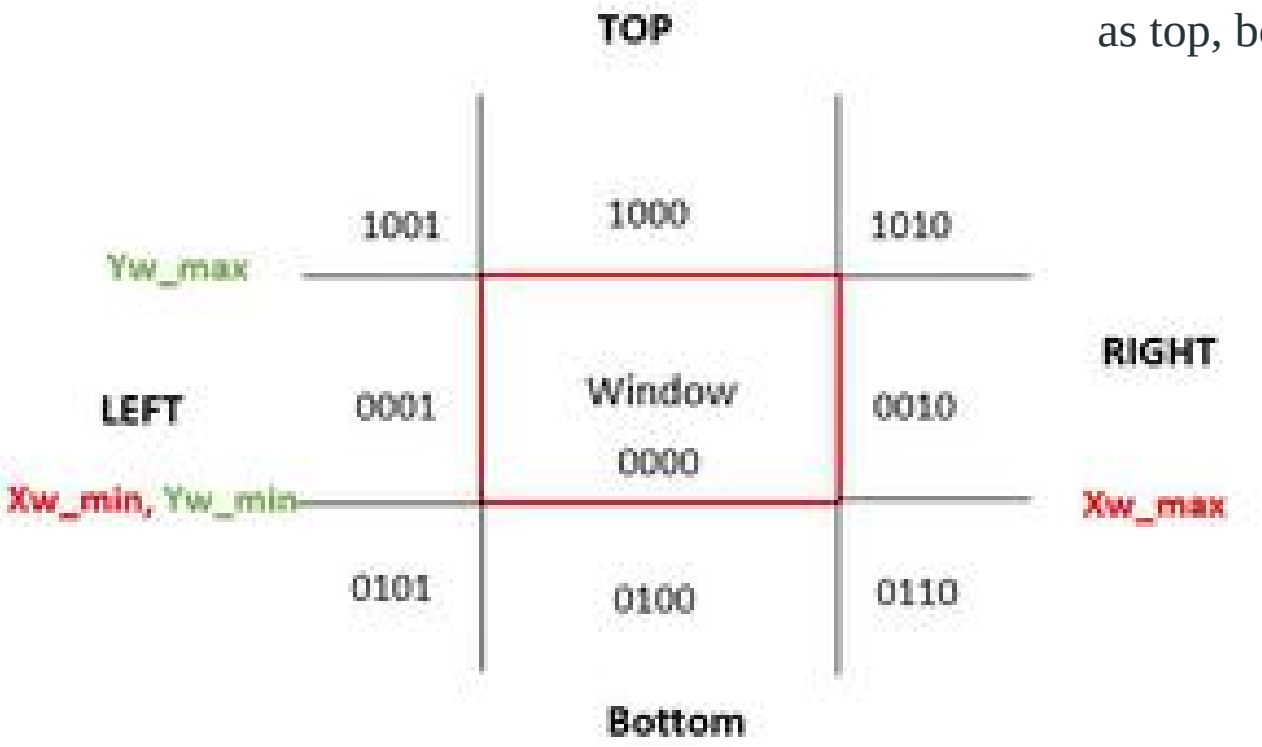
Line AB is the visible case
Line OP is an invisible case
Line PQ is an invisible line
Line IJ are clipping candidates
Line MN are clipping candidate
Line CD are clipping candidate

❑ Line Clipping:

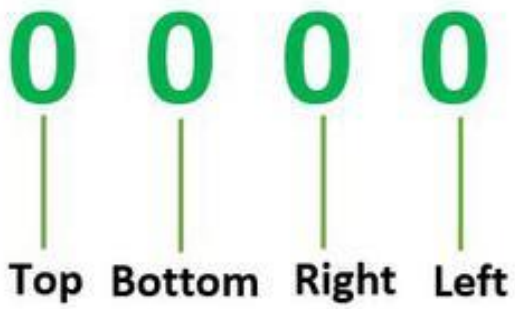
It is performed by using the line clipping algorithm. The line clipping algorithms are:

1. Cohen Sutherland Line Clipping Algorithm
2. Midpoint Subdivision Line Clipping Algorithm
3. Liang-Barsky Line Clipping Algorithm

❑ Cohen Sutherland Line Clipping Algorithm



<u>Formula to check binary digits</u>:- TBRL which can be defined as top, bottom, right, and left accordingly.



❑ Algorithm of Cohen Sutherland Line Clipping:

Step1:Calculate positions of both endpoints of the line

Step2:Perform OR operation on both of these end-points

Step3:If the OR operation gives 0000

Then

line is considered to be visible

else

Perform AND operation on both endpoints

If And $\neq$ 0000

then the line is invisible

else

And=0000

Line is considered the clipped case.

Step4:If a line is clipped case, find an intersection with boundaries of the window

m=(y2-y1)/(x2-x1)

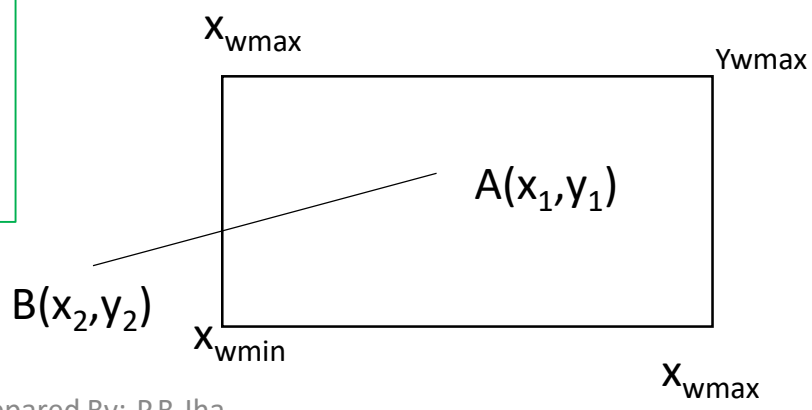
(a) line intersects with left boundary of rectangle window

$$m=\frac{y2-y1}{x2-x1}=\frac{y-y1}{x_{wmin}-x1}$$

where $X = X_{wmin}$

where X_{wminis} the minimum value of X co-ordinate of window

$$y = (y_1+m(x_{wmin}+x_1)$$



(b) line intersect with right boundary

y=y1+m(X-X1)

where $X = X_{wmax}$

where X more is maximum value of X co-ordinate of the window

(c) line intersects with bottom boundary

$$m = \frac{y2-y1}{x2-x1} = \frac{y_{wmin}-y1}{x2-x1}$$

$X=X_1+(y_{wmin}- y1)/m$

where $y = y_{wmin}$

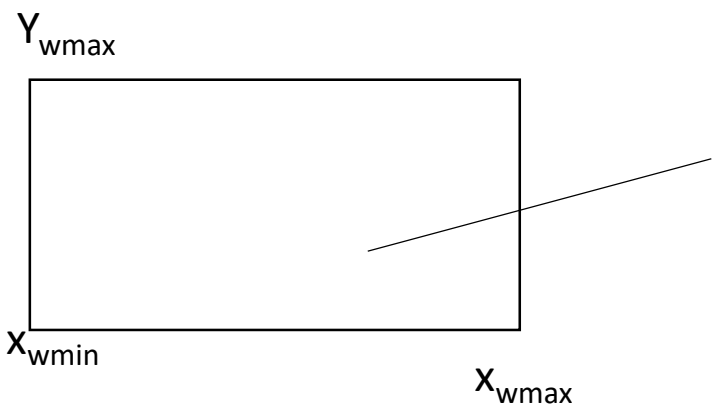
y_{wmin} is the minimum value of Y co-ordinate of the window

(d) line intersects with the top boundary

$X=X_1+(y_{wmax}-y1)/m$

where $y = {}_{ywmax}$

y_{wmax} is the maximum value of Y co-ordinate of the window



Inside the window: 0000
Left of the window: 0001
Right of the window: 0010
Below the window: 0100
Above the window: 1000

How to calculate bit to end point of line

- The region code for point (x, y) is set according to the scheme

Bit 1 = sign $(y-y_{max})$

Bit 2 = sign $(y_{min}-y)$

Bit 3 = sign $(x-x_{max})$

Bit 4 = sign $(x_{min}-x)$

$$\text{sign}(a) = \begin{cases} 1 & \text{if a is positive} \\ 0 & \text{otherwise} \end{cases}$$

window whose lower left-hand corner is at L (-3, 1) and upper right-hand corner is at R (2, 6). Find the region codes for the endpoints

a. A(-4,2) B (-1, 7)
b. C (-1, 5) D (3, 8)
c. E (-2, 3) F (1, 2)

Solution :-
For end point A(-4,2)
Assumption x = -4, y =2 x_{min} =-3,y_{min} =1, x_{max} =2 ,y_{max}=6

For bit 1
sing$(y-y_{max})$ = sing(2-6) =sign(-4) hence first bit is 0
For bit 2
sign $(y_{min}-y)$ = sign(1-2)) =sign(-1) hence second bit is 0
For bit 3
sign $(x-x_{max})$ = sign(-4-2) = sing(--6) hence 3rd bit is 0

sign $(x_{min}-x)$ = sign(-3-(-4)) =sign(=1) hence 4 bit is 1

Since A point has region code is (0001)

Similar for all end point

Class work

A (-4, 2)→ 0001	F (1, 2)→ 0000
B (-1, 7) → 1000	G (1, -2) →0100
C (-1, 5)→ 0000	H (3, 3) → 0100
D (3, 8) → 1010	I (-4, 7) → 1001
E (-2, 3) → 0000	J (-2, 10) → 1000

5) Now, overwrite the endpoints with a new one and update it.

6) Repeat the 4th step till your line doesn't get completely clipped

Let's take an example:

Clip a line A (-1,5) and B (3,8) using the Cohen Sutherland algorithm with window coordinates (-3,1) and (2,6).

Lower bound $(x_{min}, y_{min}) = (-3, 1)$
Upper bound $(x_{max}\ y_{max}) = (2, 6)$

Solution :-

- We first need to calculate the region codes for both endpoints:

Bit 1 = sign $(y - y_{max})$

Bit 2 = sign $(y_{min} - y)$

Bit 3 = sign $(x - x_{max})$

Bit 4 = sign $(x_{min} - x)$

Class work

Here xmin =-3 and ymin=1 with xmax =1 and ymax=6 with x1=-1, y1=5 and x2=3, y2=8.

S-1: Find the region code of the line point x1=-1,y1=5 as follows

– Bit 1 = Sign of (y-ymax) = Sign of (5-6) = Negative, so bit 1 would be 0.

– Bit 2 = Sign of (ymin-y) = Sign of (1-6) = Negative, so bit 2 would be 0.

– Bit 3 = Sign of (x-xmax) = Sign of (-1-1) = Negative, so bit 3 would be 0.

– Bit 4 = Sign of (xmin-x) = Sign of (-3-(-1)) = Negative, so bit 4 would be 0.

FIND THE REGION CODE OF THE LINE POINT X2=3,Y2=8 AS FOLLOWS.

– Bit 1 = Sign of (y-ymax) = Sign of (8-6) = Positive, so bit 1 would be 1.

– Bit 2 = Sign of (ymin-y) = Sign of (1-8) = Negative, so bit 2 would be 0.

– Bit 3 = Sign of (x-xmax) = Sign of (3-1) = Positive, so bit 3 would be 1.

– Bit 4 = Sign of (xmin-x) = Sign of (-3-3) = Negative, so bit 4 would be 0.

The (x1,y1) has the code (0000) and (x2,y2) has the code (1010). *Remember the codes are written from left to right. Left is bit-1.*

S-2 : Which category this line belongs to: Find, does it require clipping?

– Logical AND of both the region codes are (0000) * (1010) = (0*1, 0*0 ,0*1,0*0) = (0000). Since the logical AND of codes is zero, so the line belongs to third category- Clipping category =>Computer would clip it.

S-3: The line "Needs Clipping" based on the calculation above.

S-4: Since Bit 1 is one therefore intersection point is y=ymax and x=x1+(y1-ymax)/m. => y=6 and x= -1+(6-5)/(3/4) => -1+4/3 => 1/3. So the intersection point would be (x=1/3,y=6).

S-5: Find the region code of newfound point, C(x=1/3 and y=6) =>(0000)

Since the starting point, A(-1,5) has (0000) and new Point C is also (0000) that means both endpoints are visible. The line does not need more clipping. The algorithm would stop here.

Example -2

Let ABCD be the rectangle window with A(20,20),B(90,20),C(90,70) and D(20,70) find the region code for the ends points and use cohen sutherlands algorithm to clip the line P1P2 with P1(10,30) and P2(80,90)

❑ Liang-Barsky Line Clipping

- Liang-Barsky Line Clipping is another line clipping algorithm used to efficiently clip a line against a rectangular window. Similar to Cohen-Sutherland,
- it calculates the intersection points of the line with each window boundary and then discards the parts of the line that lie outside the window.
- However, Liang-Barsky uses parametric equations to calculate these intersection points, which often results in fewer calculations compared to Cohen-Sutherland.

The Liang-Barsky algorithm works as follows:

1. Calculate the parameters p1, p2, p3, and p4 for the line equation using the endpoints (x1, y1) and (x2, y2):

   p1 = -(x2 - x1)

   p2 = (x2 - x1)

   p3 = -(y2 - y1)

   p4 = (y2 - y1)

2. Calculate the values of q1, q2, and q3 for each window boundary (xmin, ymin) and (xmax, ymax):

   q1 = x1 - xmin

   q2 = xmax - x1

   q3 = y1 – ymin

   q4 = ymax - y1

3. Determine the parameter values t1 and t2:

   t1 = max(0, max(q1 / p1, q3 / p3))

   t2 = min(1, min(q2 / p2, q4 / p4))

4. Check if the line is outside the window ($t1 > t2$). If so, the line is completely outside and can be rejected.

5. If $t1 \leq t2$, calculate the clipped endpoints ($x1'$, $y1'$) and ($x2'$, $y2'$):

$$x1' = x1 + t1 * (x2 - x1)$$

$$y1' = y1 + t1 * (y2 - y1)$$

$$x2' = x1 + t2 * (x2 - x1)$$

$$y2' = y1 + t2 * (y2 - y1)$$

6. The line segment defined by ($x1'$, $y1'$) and ($x2'$, $y2'$) is the clipped portion of the original line inside the rectangular window.

Let's go through an example:

Consider a line segment defined by two endpoints P1(30, 50) and P2(90, 110). We want to clip this line against a rectangular window with boundaries: (xmin, ymin) = (50, 70) and (xmax, ymax) = (80, 100).

Step 1: Calculate the parameters:
p1 = -(90 - 30) = -60
p2 = 90 - 30 = 60
p3 = -(110 - 50) = -60
p4 = 110 - 50 = 60

Step 2: Calculate the values of q1, q2, q3, and q4 for each window boundary:
q1 = 30 - 50 = -20
q2 = 80 - 30 = 50
q3 = 50 - 70 = -20
q4 = 100 - 50 = 50

Step 3: Determine the parameter values t1 and t2:
t1 = max(0, max(-20 / -60, -20 / -60)) = max(0, max(1/3, 1/3)) = 1/3
t2 = min(1, min(50 / 60, 50 / 60)) = min(1, min(5/6, 5/6)) = 5/6

Step 4: Check if the line is outside the window:
t1 (1/3) ≤ t2 (5/6) -> The line is inside or partially inside the window

Step 5: Calculate the clipped endpoints:
x1' = 30 + (1/3) * (90 - 30) = 50
y1' = 50 + (1/3) * (110 - 50) = 70
x2' = 30 + (5/6) * (90 - 30) = 80
y2' = 50 + (5/6) * (110 - 50) = 100

Step 6: The clipped line segment is defined by the endpoints P1'(50, 70) and P2'(80, 100)

So, the clipped line segment (P1' to P2') represents the portion of the original line that lies inside the rectangular window (50, 70) to (80, 100) using the Liang-Barsky Line Clipping algorithm.

❑Polygon Clipping(Sutherland Hodgeman polygon clipping)

- Sutherland-Hodgman polygon clipping is a technique used to clip a polygon against an arbitrary clipping window.
- The algorithm takes a subject polygon and a clipping window defined by its boundary edges and outputs the portion of the subject polygon that lies inside the clipping window.

The steps of the Sutherland-Hodgman polygon clipping algorithm are as follows:

1. Initialize an empty list to store the clipped vertices of the subject polygon.

2. Iterate through each edge of the clipping window in a clockwise manner.

3. For each clipping edge, perform the following operations on the subject polygon:

   a. Initialize a new list to store the clipped vertices after clipping against the current clipping edge.

   b. Iterate through each consecutive pair of vertices (V1, V2) in the subject polygon.

.

c. Classify V1 and V2 with respect to the current clipping edge (inside or outside).

d. If both V1 and V2 are inside the clipping edge, add V2 to the new list.

e. If V1 is inside and V2 is outside, add the intersection point of the edge (V1, V2) with the clipping edge to the new list.

f. If V1 is outside and V2 is inside, add the intersection point and V2 to the new list.

g. If both V1 and V2 are outside, do not add anything to the new list.

h. After iterating through all the vertices of the subject polygon, update the subject polygon with the newly clipped vertices.

4. After iterating through all the clipping edges, the subject polygon will contain the clipped portion.

Let's go through an example:

Consider a subject polygon defined by its vertices: A(50, 100), B(100, 150), C(150, 100), and D(100, 50). We want to clip this polygon against a rectangular clipping window with boundaries: (xmin, ymin) = (75, 75) and (xmax, ymax) = (125, 125).

Step 1: Initialize an empty list for storing the clipped vertices.

Step 2: Iterate through each edge of the clipping window (clockwise):

a. Clipping edge: (75, 75) to (75, 125)

b. Clipping edge: (75, 125) to (125, 125)

c. Clipping edge: (125, 125) to (125, 75)

d. Clipping edge: (125, 75) to (75, 75)

Step 3: Clipping against each edge:

a. Clipping edge: (75, 75) to (75, 125)
The new list after clipping against this edge:
New list: B'(75, 100), C'(75, 100)

b. Clipping edge: (75, 125) to (125, 125)
The new list after clipping against this edge:
New list: B"(100, 125), C"(100, 125)

c. Clipping edge: (125, 125) to (125, 75)
The new list after clipping against this edge:
New list:

d. Clipping edge: (125, 75) to (75, 75)
The new list after clipping against this edge:
New list: D'(100, 75), A'(100, 75)

Step 4: The final clipped polygon is defined by the vertices: A'(100, 75), B"(100, 125), C"(100, 125), D'(100, 75).

These vertices represent the portion of the original subject polygon that lies inside the rectangular clipping window (75, 75) to (125, 125) after applying the Sutherland-Hodgman polygon clipping algorithm.

How to assign bit to end point of line

- The region code for point (x, y) is set according to the scheme

Bit 1 = sign $(y-y_{max})$

Bit 2 = sign $(y_{min}-y)$

Bit 3 = sign $(x-x_{max})$

Bit 4 = sign $(x_{min}-x)$

$$\text{sign}(a) = \begin{cases} 1 & \text{if a is positive} \\ 0 & \text{otherwise} \end{cases}$$

window whose lower left-hand corner is at L (-3, 1) and upper right-hand corner is at R (2, 6). Find the region codes for the endpoints

a. A(-4,2) B (-1, 7)
b. C (-1, 5) D (3, 8)
c. E (-2, 3) F (1, 2)

Solution :-
For end point A(-4,2)
Assumption x = -4, y =2 x_{min} =-3,y_{min} =1, x_{max} =2 ,y_{max}=6

For bit 1
sing$(y-y_{max})$ = sing(2-6) =sign(-4) hence first bit is 0
For bit 2
sign $(y_{min}-y)$ = sign(1-2)) =sign(-1) hence second bit is 0
For bit 3
sign $(x-x_{max})$ = sign(-4-2) = sing(--6) hence 3rd bit is 0

sign $(x_{min}-x)$ = sign(-3-(-4)) =sign(=1) hence 4 bit is 1

Since A point has region code is (0001)

Similar for all end point

Class work

A (-4, 2)→ 0001	F (1, 2)→ 0000
B (-1, 7) → 1000	G (1, -2) →0100
C (-1, 5)→ 0000	H (3, 3) → 0100
D (3, 8) → 1010	I (-4, 7) → 1001
E (-2, 3) → 0000	J (-2, 10) → 1000